

**ĐẠI HỌC BÁCH KHOA
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



BÁO CÁO THỰC TẬP

Đơn vị thực tập: Viettel High Tech

Sinh viên thực hiện:	Author Name
Mã số sinh viên:	
Lớp:	
Khoa:	Khoa học và Kỹ thuật Máy tính
Giảng viên hướng dẫn:	
Người hướng dẫn tại đơn vị:	

Thành phố Hồ Chí Minh, Ngày 19 tháng 8 năm 2026

Mục lục

1	Mở đầu	5
1.1	Bối cảnh thực tập	5
1.2	Mục tiêu và phạm vi báo cáo	5
2	Cơ sở lý thuyết về Basic C	5
2.1	Quá trình hình thành chương trình C	5
2.2	Liên kết tĩnh và liên kết động	6
3	Memory layout của chương trình C	7
3.1	Không gian địa chỉ ảo của process	7
3.2	Thời điểm compile-time, load-time và run-time	7
4	Tổng quan về Linux Programming	8
4.1	Kiến trúc GNU/Linux và ranh giới user space – kernel space	8
4.2	System call	8
5	Từ kiến trúc UNIX đến tổ chức Linux Kernel	8
5.1	Hai trụ cột của mô hình UNIX: file và process	8
5.2	Phân chia các phân hệ trong kernel	9
5.3	File descriptor, file table và inode	9
5.4	Cache dữ liệu và cân bằng giữa hiệu năng với tính nhất quán	10
5.5	Interrupt, exception và context thực thi	10
5.6	Ý nghĩa đối với việc đọc và debug Linux	10
6	Virtual memory trong Linux	11
6.1	Khái niệm bộ nhớ ảo	11
6.2	Page, page table và MMU	11
6.3	Page fault và demand paging	12
6.4	Memory mapping và chia sẻ dữ liệu	12

6.5	Ý nghĩa đối với Embedded Linux	12
7	Process trong Linux	13
7.1	Khái niệm process và tài nguyên của process	13
7.2	Trạng thái của process	13
7.3	Tạo process bằng fork	14
7.4	exec và wait	14
7.5	Daemon và service trong hệ thống nhúng	14
8	Thread trong Linux	15
8.1	Khái niệm thread	15
8.2	So sánh process và thread	15
8.3	Race condition và critical section	15
8.4	Deadlock, condition variable và semaphore	16
8.5	Ứng dụng thread trong Embedded Linux	16
9	Interprocess Communication	16
9.1	Nhu cầu giao tiếp liên tiến trình	16
9.2	Pipe và FIFO	17
9.3	Message queue	17
9.4	Shared memory	17
9.5	Semaphore và đồng bộ process	18
9.6	Socket như một cơ chế IPC	18
9.7	Lựa chọn IPC trong thiết kế nhúng	18
10	Signal trong Linux	18
10.1	Khái niệm signal	18
10.2	Một số signal thường gặp	19
10.3	Xử lý signal bằng signal và sigaction	19
10.4	Signal và quản lý process con	19

10.5	Signal trong daemon Embedded Linux	19
11	Linux Device Driver và giao tiếp với phần cứng	20
11.1	Vai trò của device driver	20
11.2	Module kernel và vòng đời driver	20
11.3	Char, block và network driver	21
11.4	file_operations và đường đi của lời gọi hệ thống	21
11.5	Concurrency và bảo vệ dữ liệu trong kernel	21
11.6	I/O memory, thanh ghi và memory barrier	22
11.7	Interrupt và deferred work	22
11.8	DMA và quản lý buffer	22
11.9	Linux device model, bus, device và driver	23
11.10	Debug driver và liên hệ MediaTek AP	23
12	Git và quy trình làm việc nhóm	24
13	Socket Programming bằng C trên Linux	24
13.1	Vai trò của socket trong Linux Programming	24
13.2	TCP socket và UDP socket	24
13.3	Luồng hoạt động của chương trình server và client	25
13.4	Những vấn đề cần chú ý khi lập trình socket C	25
14	OpenWrt trong hệ sinh thái Embedded Linux	26
14.1	Tổng quan về OpenWrt	26
14.2	Kiến trúc phần mềm của OpenWrt	26
14.3	Phát triển ứng dụng C trên OpenWrt	27
14.4	Liên hệ OpenWrt với boot sequence và hệ thống nhúng	27
15	Boot sequence, bootloader và kiến trúc ARM trong Embedded Linux	28
15.1	Vai trò của bootloader trong hệ thống nhúng	28
15.2	Đặc điểm boot trên kiến trúc ARM	28

15.3 Phase 1: ROM code trong SoC	29
15.4 Phase 2: SPL và khởi tạo DRAM	29
15.5 Phase 3: TPL hoặc U-Boot đầy đủ	29
15.6 Nạp kernel, initramfs và Device Tree	30
15.7 Handoff từ bootloader sang Linux Kernel	30
15.8 Falcon mode và tối ưu thời gian boot	31
15.9 Tổng kết luồng boot trên ARM Embedded Linux	31
16 Ứng dụng kiến thức vào MediaTek AP 6	32
16.1 Bối cảnh áp dụng trên thiết bị Access Point	32
16.2 Cấu hình Wi-Fi SoftAP bằng profile và iwpriv	32
16.3 Các nhóm tham số quan trọng khi triển khai AP 6	33
16.4 Liên hệ với socket programming và daemon quản lý	33
16.5 Quy trình áp dụng vào bring-up và kiểm thử	34
17 Văn hóa công ty trong quá trình thực tập	34
17.1 Môi trường làm việc và tinh thần học hỏi	34
17.2 Văn hóa seminar và chia sẻ kiến thức cho intern	35
17.3 Tác phong và bài học cá nhân	35
18 Nhật ký công việc	35
19 Kết quả đạt được	36
20 Khó khăn và hướng khắc phục	37
21 Kết luận	37

1 Mở đầu

1.1 Bối cảnh thực tập

Embedded Linux là một hướng phát triển quan trọng trong lĩnh vực hệ thống nhúng hiện đại. Khác với lập trình ứng dụng thông thường, lập trình trên nền tảng Embedded Linux yêu cầu người học hiểu đồng thời cả phần mềm hệ thống, ngôn ngữ C, kiến trúc Linux và một số đặc điểm phần cứng của thiết bị. Một ứng dụng nhúng chạy trên Linux không tồn tại độc lập, mà phụ thuộc vào quá trình biên dịch, thư viện hệ thống, kernel, driver, root filesystem, bootloader và cơ chế quản lý tài nguyên của hệ điều hành.

Trong quá trình thực tập, việc học Basic C và Linux Programming được xem là bước chuẩn bị bắt buộc. Basic C giúp người học hiểu cách source code được chuyển thành chương trình thực thi, cách bộ nhớ của chương trình được tổ chức và cách thư viện được liên kết. Linux Programming giúp người học hiểu cách chương trình user space tương tác với kernel thông qua system call, cách Linux quản lý process, thread, bộ nhớ ảo, signal và các cơ chế giao tiếp liên tiến trình. Những kiến thức này không chỉ phục vụ việc viết chương trình C trên Linux, mà còn là cơ sở để phân tích lỗi, tối ưu tài nguyên và xây dựng phần mềm ổn định trên thiết bị nhúng.

1.2 Mục tiêu và phạm vi báo cáo

Mục tiêu của báo cáo là hệ thống hóa lại các nội dung đã tìm hiểu trong giai đoạn thực tập, đồng thời liên hệ chúng với bài toán Embedded Linux. Báo cáo hướng đến việc giải thích bản chất thay vì chỉ liệt kê khái niệm. Vì vậy, mỗi phần đều được trình bày theo mạch từ lý thuyết cơ bản, vai trò trong Linux, đến ý nghĩa khi áp dụng vào hệ thống nhúng.

Phạm vi báo cáo tập trung vào nền tảng Basic C, Linux Programming, kiến trúc UNIX/Linux kernel, virtual memory, process, thread, IPC, signal, device driver, socket programming, OpenWrt, ứng dụng trên MediaTek AP 6, boot sequence và một số ghi nhận về văn hóa công ty trong quá trình thực tập. Phần device driver trình bày các nguyên lý và đường đi dữ liệu quan trọng, chưa đi sâu đến mức triển khai hoàn chỉnh một driver cho board cụ thể. Các nội dung porting board, xây dựng root filesystem bằng Buildroot/Yocto hoặc tối ưu kernel được xem là hướng phát triển tiếp theo.

2 Cơ sở lý thuyết về Basic C

2.1 Quá trình hình thành chương trình C

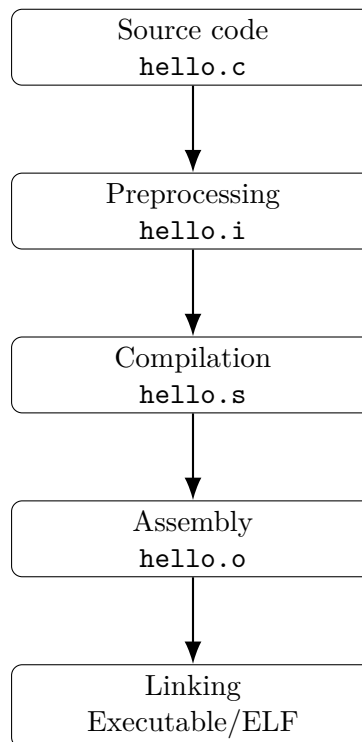
Một chương trình C không được CPU thực thi trực tiếp từ file mã nguồn. Trước khi chạy được trên hệ thống, mã nguồn phải trải qua một chuỗi xử lý gồm tiền xử lý, biên dịch, hợp dịch và liên kết. Chuỗi này thường được che giấu phía sau một lệnh đơn giản như `gcc hello.c -o hello`, nhưng về bản chất mỗi giai đoạn đảm nhiệm một vai trò riêng.

Ở giai đoạn tiền xử lý, bộ tiền xử lý `cpp` xử lý các chỉ thị bắt đầu bằng ký tự `#`. Các chỉ thị như `#include`, `#define`, `#ifdef` được mở rộng để tạo ra file trung gian. Ví dụ, lệnh `gcc -E`

`test_pp.c -o test_pp.i` cho phép quan sát kết quả sau tiền xử lý. Đối với Embedded Linux, giai đoạn này đặc biệt quan trọng vì nhiều dự án dùng macro để bật tắt tính năng, chọn cấu hình phần cứng hoặc phân biệt môi trường build cho từng board.

Sau tiền xử lý, compiler dịch file trung gian sang mã assembly. Assembly là dạng biểu diễn gần với kiến trúc CPU hơn so với ngôn ngữ C. Việc quan sát file assembly bằng lệnh như `gcc -S hello.i -o hello.s` giúp người học hiểu cách lệnh C được ánh xạ thành thao tác ở mức thấp, từ đó có thêm cơ sở khi phân tích hiệu năng, stack frame hoặc calling convention.

Tiếp theo, assembler dịch mã assembly thành object file. Object file chứa mã máy dạng relocatable, nghĩa là nó đã có mã nhị phân nhưng chưa phải chương trình hoàn chỉnh. Một chương trình thực tế thường gồm nhiều file nguồn, nhiều object file và có thể phụ thuộc vào thư viện bên ngoài. Vì vậy, linker cần kết hợp các object file, giải quyết symbol và tạo ra file thực thi cuối cùng ở định dạng phù hợp, thường là ELF trên Linux.



2.2 Liên kết tĩnh và liên kết động

Liên kết là giai đoạn có ảnh hưởng trực tiếp đến cách chương trình được phân phối và chạy trên hệ thống. Với liên kết tĩnh, mã thư viện cần thiết được đưa trực tiếp vào file thực thi. Cách này làm chương trình ít phụ thuộc hơn vào môi trường runtime, nhưng kích thước file thực thi thường lớn. Trong hệ thống nhúng có root filesystem tối giản, static linking đôi khi hữu ích vì chương trình có thể chạy ngay cả khi thiếu một số shared library.

Ngược lại, liên kết động chỉ lưu thông tin tham chiếu đến thư viện dùng chung. Khi chương trình được chạy, dynamic loader sẽ nạp các shared library cần thiết vào bộ nhớ. Cách này giúp giảm kích

thước executable, tiết kiệm bộ nhớ khi nhiều process dùng chung thư viện và thuận tiện cho việc cập nhật thư viện. Tuy nhiên, chương trình sẽ phụ thuộc vào việc thư viện đúng phiên bản có tồn tại trong root filesystem hay không. Vì vậy, khi phát triển Embedded Linux, lập trình viên cần hiểu rõ lựa chọn linking để tránh lỗi thiếu thư viện khi triển khai lên board thật.

3 Memory layout của chương trình C

3.1 Không gian địa chỉ ảo của process

Memory layout của một chương trình Linux không nên được hiểu là cách chương trình chiếm trực tiếp RAM vật lý. Thực tế, mỗi process hoạt động trong một không gian địa chỉ ảo riêng. Không gian này được hệ điều hành và phần cứng MMU ánh xạ sang bộ nhớ vật lý khi cần thiết. Cơ chế này tạo cảm giác mỗi process có một vùng nhớ liên tục và độc lập, đồng thời giúp kernel bảo vệ process này khỏi việc truy cập trái phép vào vùng nhớ của process khác.

Trong không gian địa chỉ ảo, chương trình thường được chia thành nhiều vùng. Vùng text chứa mã lệnh thực thi của chương trình. Vùng dữ liệu chỉ đọc chứa hằng số. Vùng data chứa biến toàn cục hoặc biến static đã được khởi tạo. Vùng BSS chứa biến toàn cục hoặc static chưa khởi tạo hoặc khởi tạo bằng 0. Heap được dùng cho cấp phát động thông qua các hàm như `malloc` và `free`. Stack được dùng cho biến cục bộ, tham số hàm, địa chỉ trả về và stack frame. Ngoài ra còn có vùng memory mapping dùng để ánh xạ file, shared library hoặc shared memory.

Việc hiểu memory layout giúp giải thích nhiều lỗi phổ biến trong C. Nếu chương trình truy cập con trỏ sai, ghi vượt mảng hoặc dùng vùng nhớ đã giải phóng, kernel có thể phát hiện vi phạm quyền truy cập và gửi `SIGSEGV`. Nếu chương trình cấp phát heap nhưng không giải phóng, lỗi memory leak có thể xuất hiện, đặc biệt nguy hiểm trong hệ thống nhúng chạy liên tục trong thời gian dài. Nếu hàm đệ quy quá sâu hoặc cấp phát biến cục bộ quá lớn, stack overflow có thể làm chương trình dừng bất thường.

3.2 Thời điểm compile-time, load-time và run-time

Có thể nhìn chương trình ở ba thời điểm khác nhau. Tại compile-time và link-time, compiler kiểm tra cú pháp, dịch source code thành object code, còn linker kết hợp các object file và thư viện để tạo executable. Tại load-time, khi người dùng chạy chương trình, OS loader đọc file thực thi từ thiết bị lưu trữ và ánh xạ các segment vào không gian địa chỉ ảo của process. Tại run-time, chương trình thực sự thực thi hàm `main`, cấp phát bộ nhớ, gọi hàm, xử lý dữ liệu, tạo process hoặc thread và gọi system call để yêu cầu dịch vụ từ kernel.

Sự phân biệt này rất quan trọng trong quá trình debug. Một lỗi unresolved symbol thường thuộc giai đoạn linking. Một lỗi thiếu shared library thường xuất hiện tại load-time. Một lỗi segmentation fault, deadlock hoặc race condition thường xuất hiện tại run-time. Đối với Embedded Linux, việc xác định lỗi thuộc giai đoạn nào giúp rút ngắn thời gian phân tích khi chương trình chạy đúng trên máy phát triển nhưng lỗi trên board mục tiêu.

4 Tổng quan về Linux Programming

4.1 Kiến trúc GNU/Linux và ranh giới user space – kernel space

Linux kernel là phần lõi của hệ điều hành. Kernel chịu trách nhiệm quản lý CPU, bộ nhớ, filesystem, network stack, thiết bị ngoại vi và driver. Phía trên kernel là user space, nơi các ứng dụng người dùng và thư viện như glibc hoạt động. Các chương trình do lập trình viên viết thường nằm trong user space và không được phép truy cập trực tiếp tài nguyên phần cứng hoặc vùng nhớ của kernel.

Sự tách biệt giữa user space và kernel space là cơ chế bảo vệ quan trọng. Nếu mọi chương trình đều có thể truy cập trực tiếp phần cứng hoặc bộ nhớ kernel, một lỗi nhỏ trong ứng dụng cũng có thể làm treo toàn bộ hệ thống. Nhờ cơ chế phân quyền của CPU và kernel, lỗi trong một process thường chỉ làm process đó dừng, trong khi phần còn lại của hệ thống vẫn tiếp tục hoạt động.

4.2 System call

Khi chương trình user space cần đọc file, ghi dữ liệu ra thiết bị, tạo process, cấp phát ánh xạ bộ nhớ hoặc giao tiếp mạng, nó phải yêu cầu kernel thực hiện thông qua system call. System call là điểm vào được kiểm soát từ user space xuống kernel space. Khi system call được gọi, CPU chuyển từ user mode sang kernel mode, kernel kiểm tra tham số, thực hiện service routine tương ứng và trả kết quả về chương trình.

Các lời gọi như `open`, `read`, `write`, `close`, `fork`, `exec`, `wait`, `mmap`, `pipe`, `socket`, `kill` và `sigaction` là những ví dụ thường gặp. Trong Embedded Linux, nhiều thiết bị ngoại vi được biểu diễn dưới dạng file trong `/dev`. Vì vậy, thao tác đọc ghi file thực chất có thể là thao tác với UART, GPIO, SPI, I2C hoặc một driver thiết bị. Điều này cho thấy Linux Programming là cầu nối trực tiếp giữa ứng dụng C và phần cứng trong hệ thống nhúng.

5 Từ kiến trúc UNIX đến tổ chức Linux Kernel

5.1 Hai trụ cột của mô hình UNIX: file và process

Thiết kế UNIX tổ chức hệ điều hành quanh hai khái niệm trung tâm là file và process. File biểu diễn dữ liệu hoặc một điểm giao tiếp I/O, còn process biểu diễn một chương trình đang có “đời sống” trong hệ thống. Cách nhìn này tạo ra một mô hình tương đối thống nhất: ứng dụng mở một đối tượng bằng `open`, nhận về file descriptor, sau đó dùng `read`, `write`, `ioctl` hoặc `close` để làm việc với đối tượng đó. Đối tượng phía sau có thể là file thường, terminal, pipe, socket hoặc device node.

Điểm quan trọng không phải mọi đối tượng đều giống hệt file trên đĩa, mà là kernel cung cấp một giao diện chung để user space không cần biết toàn bộ chi tiết phần cứng. Trên Embedded Linux, cùng một chương trình C có thể đọc UART qua một file trong `/dev`, nhận dữ liệu mạng qua socket và đọc cấu hình từ filesystem bằng những lời gọi hệ thống có cách sử dụng gần nhau. Đây là lý do triết lý UNIX vẫn có giá trị lớn đối với phần mềm nhúng hiện đại.

Process là đơn vị mà kernel dùng để phân biệt các chương trình đang chạy, cấp phát không gian địa

chỉ, quản lý quyền, lưu trạng thái CPU và theo dõi các tài nguyên đang mở. Khi process gọi system call, kernel thực thi thay mặt process đó trong kernel mode. Kernel không phải một chương trình user space chạy song song với các process; phần lớn hoạt động kernel diễn ra trong ngữ cảnh của process hiện tại hoặc trong ngữ cảnh xử lý interrupt. Cách nhìn này giúp giải thích vì sao một lời gọi `read` có thể làm process ngủ, driver xử lý thiết bị, interrupt đánh thức process và system call sau đó mới trả về user space.

5.2 Phân chia các phân hệ trong kernel

Kiến trúc UNIX kinh điển có thể được nhìn thành các phân hệ chính gồm system call interface, file subsystem, process control, memory management, scheduler, IPC và hardware control. Linux kế thừa tư tưởng phân chia trách nhiệm này nhưng mở rộng thành một kernel lớn, có tính module và hỗ trợ nhiều kiến trúc CPU, filesystem, network protocol và loại thiết bị.

System call interface là cổng vào có kiểm soát từ user space. Process control quản lý tạo, kết thúc, ngủ, thức và chuyển trạng thái process. Scheduler lựa chọn task nào được sử dụng CPU. Memory management quản lý không gian địa chỉ ảo, page table, page cache và cấp phát bộ nhớ kernel. Filesystem layer quản lý namespace, quyền truy cập và ánh xạ thao tác file xuống filesystem cụ thể. Device driver chuyển yêu cầu trừu tượng của kernel thành thao tác với thanh ghi, buffer, bus và interrupt của phần cứng.

Các phân hệ này không hoạt động độc lập hoàn toàn. Khi ứng dụng đọc một device node, system call interface chuyển yêu cầu xuống VFS, VFS tìm các operation của driver, driver có thể đưa process vào trạng thái ngủ nếu chưa có dữ liệu, interrupt handler ghi nhận dữ liệu đến và đánh thức process, sau đó scheduler cho process tiếp tục. Một thao tác tương tự đơn giản ở user space vì vậy có thể đi qua nhiều tầng của kernel.

5.3 File descriptor, file table và inode

Trong mô hình UNIX, file descriptor chỉ là một số nguyên nhỏ có ý nghĩa trong phạm vi một process. Nó là chỉ số dẫn đến bảng file đang mở của process. Entry tương ứng tiếp tục tham chiếu đến đối tượng file trong kernel, nơi lưu vị trí đọc ghi hiện tại, cờ mở file và các operation được phép thực hiện. Đối với file trên filesystem, đối tượng này liên hệ với inode chứa metadata của file.

Inode không lưu tên file. Nó lưu thông tin như loại file, quyền truy cập, chủ sở hữu, kích thước, số hard link và vị trí dữ liệu. Tên file nằm trong directory entry và được kernel phân giải thành inode khi xử lý pathname. Việc tách tên khỏi inode giải thích một số hành vi đặc trưng của UNIX: nhiều tên có thể trỏ đến cùng một inode, và một file đã bị `unlink` vẫn có thể tiếp tục được process đọc nếu file descriptor của nó còn mở. Dữ liệu chỉ thực sự được giải phóng khi không còn directory entry phù hợp và không còn tham chiếu đang mở.

Linux hiện đại đặt một lớp Virtual File System giữa system call và filesystem cụ thể. VFS cho phép `open`, `read` và `write` hoạt động thống nhất trên ext4, tmpfs, procfs, sysfs hoặc device node. Mỗi loại đối tượng cung cấp các operation tương ứng. Điều này đặc biệt quan trọng trong Embedded Linux vì cùng một root filesystem có thể kết hợp filesystem trên flash, filesystem tạm trong RAM, procfs để quan sát kernel và sysfs để biểu diễn device model.

5.4 Cache dữ liệu và cân bằng giữa hiệu năng với tính nhất quán

Một nguyên lý quan trọng trong thiết kế UNIX là tránh truy cập thiết bị lưu trữ cho mọi thao tác đọc ghi. Dữ liệu block được giữ trong cache để những lần truy cập sau có thể phục vụ từ RAM. Cơ chế delayed write cho phép kernel trì hoãn một số thao tác ghi và gộp chúng lại, nhờ đó giảm số lần I/O và tăng throughput. Linux hiện đại phát triển ý tưởng này thành page cache thống nhất hơn, nhưng bài toán cơ bản vẫn giống nhau: sử dụng RAM để che độ trễ của thiết bị lưu trữ.

Cache tạo ra đánh đổi giữa hiệu năng và tính bền vững dữ liệu. Khi **write** trả về, dữ liệu có thể mới chỉ nằm trong RAM và chưa được ghi xuống flash hoặc disk. Ứng dụng cần tính nhất quán cao phải dùng cơ chế đồng bộ phù hợp như **fsync** hoặc thiết kế cập nhật nguyên tử. Trên router và AP, vấn đề này liên quan trực tiếp đến lưu cấu hình, cập nhật firmware và khả năng phục hồi sau mất điện. Ghi quá thường xuyên xuống flash còn làm tăng hao mòn, trong khi trì hoãn quá lâu có thể làm mất thay đổi mới nhất.

5.5 Interrupt, exception và context thực thi

Exception thường phát sinh đồng bộ từ lệnh đang thực thi, ví dụ page fault hoặc system call, còn interrupt thường đến bất đồng bộ từ phần cứng như timer, UART, Ethernet hoặc Wi-Fi. Kernel phải lưu ngữ cảnh cần thiết, chạy routine xử lý phù hợp và sau đó quyết định tiếp tục process cũ hay lập lịch task khác.

Không phải mọi công việc đều nên thực hiện trực tiếp trong interrupt handler. Phần xử lý ở ngữ cảnh interrupt cần ngắn, không được ngủ và phải tránh giữ CPU quá lâu. Driver thường xác nhận interrupt, lấy trạng thái tối thiểu và chuyển phần việc nặng hơn sang cơ chế deferred work. Tư tưởng tách phần khẩn cấp khỏi phần có thể trì hoãn là nền tảng để giữ hệ thống phản hồi tốt khi có nhiều nguồn sự kiện đồng thời.

5.6 Ý nghĩa đối với việc đọc và debug Linux

Các tài liệu về UNIX và Linux kernel cho thấy rằng system call, process, virtual memory, filesystem và driver không phải các chương riêng biệt chỉ để học thuộc. Chúng là các lớp cùng tham gia xử lý một yêu cầu từ ứng dụng. Khi debug, cần xác định lỗi nằm ở user space, ranh giới system call, VFS, driver, interrupt hay phần cứng.

Ví dụ, một lệnh cấu hình Wi-Fi thất bại có thể do process truyền tham số sai, ioctl không được driver hỗ trợ, interface chưa được probe, firmware Wi-Fi chưa nạp, interrupt không hoạt động hoặc Device Tree mô tả sai tài nguyên. Tư duy đi theo đường đi của yêu cầu từ user space xuống kernel và quay trở lại giúp việc phân tích lỗi có hệ thống hơn so với chỉ thử lệnh ngẫu nhiên.

6 Virtual memory trong Linux

6.1 Khái niệm bộ nhớ ảo

Virtual memory là một trong những cơ chế quan trọng nhất của hệ điều hành Linux. Khi một chương trình đang chạy, nó không làm việc trực tiếp với địa chỉ vật lý trên RAM mà làm việc với địa chỉ ảo do hệ điều hành cung cấp. Mỗi process có cảm giác như nó sở hữu một không gian địa chỉ riêng, liên tục và độc lập. Trên thực tế, các địa chỉ ảo đó được kernel và phần cứng MMU chuyển đổi thành địa chỉ vật lý tương ứng khi chương trình truy cập bộ nhớ.

Cách tổ chức này giải quyết nhiều vấn đề cùng lúc. Nếu không có virtual memory, các chương trình phải chia sẻ trực tiếp RAM vật lý, dễ ghi đè lên vùng nhớ của nhau và rất khó bảo vệ hệ thống. Với virtual memory, mỗi process có không gian riêng, kernel có thể kiểm soát quyền truy cập từng vùng nhớ và có thể ánh xạ nhiều địa chỉ ảo khác nhau đến cùng một vùng vật lý khi cần chia sẻ dữ liệu. Đây là nền tảng cho memory protection, shared library, memory mapped file và cơ chế cô lập giữa các process.

Trong slide Linux Programming, virtual memory được trình bày như một mảng byte liên tục đối với từng process. Tuy nhiên, mảng byte này không nằm hoàn toàn trong RAM. Disk hoặc thiết bị lưu trữ có thể chứa toàn bộ nội dung cần thiết của chương trình, còn RAM chỉ giữ những phần đang được sử dụng. Cách làm này giúp hệ thống chạy được nhiều process hơn dung lượng RAM vật lý, đồng thời cho phép kernel nạp dữ liệu theo nhu cầu thay vì nạp tất cả ngay từ đầu.

6.2 Page, page table và MMU

Virtual memory thường được quản lý theo đơn vị page. Không gian địa chỉ ảo được chia thành nhiều trang có kích thước cố định, ví dụ 4 KiB trên nhiều kiến trúc. Địa chỉ ảo có thể được tách thành hai phần: số trang ảo và offset bên trong trang. Khi CPU truy cập một địa chỉ ảo, MMU sử dụng page table để tìm xem trang ảo đó đang ánh xạ đến frame vật lý nào trong RAM.

Page table là cấu trúc dữ liệu do kernel quản lý. Mỗi entry trong page table không chỉ chứa địa chỉ vật lý tương ứng mà còn chứa các bit trạng thái và quyền truy cập. Các bit này có thể cho biết trang có hiện diện trong RAM hay không, có được phép đọc hay ghi không, có được phép thực thi không, trang thuộc user space hay kernel space, và trang có bị sửa đổi hay chưa. Nhờ đó, kernel có thể phát hiện các lỗi như ghi vào vùng chỉ đọc, thực thi vùng không cho phép hoặc truy cập địa chỉ không hợp lệ.

Để tăng tốc quá trình dịch địa chỉ, CPU thường có TLB, tức Translation Lookaside Buffer. TLB lưu cache các ánh xạ địa chỉ ảo sang địa chỉ vật lý được dùng gần đây. Nếu không có TLB, mỗi lần truy cập bộ nhớ đều phải đi qua page table trong RAM, làm hệ thống chậm đáng kể. Khi kernel thay đổi page table, ví dụ khi context switch sang process khác hoặc khi mmap vùng nhớ mới, TLB có thể cần được cập nhật hoặc invalid để đảm bảo kết quả dịch địa chỉ đúng.

6.3 Page fault và demand paging

Page fault xảy ra khi process truy cập một trang mà MMU không thể ánh xạ ngay lập tức. Page fault không phải lúc nào cũng là lỗi nghiêm trọng. Trong cơ chế demand paging, khi chương trình bắt đầu chạy, kernel không cần nạp toàn bộ code và data vào RAM. Chỉ khi process thực sự truy cập đến một trang chưa có trong RAM, page fault mới xảy ra và kernel nạp trang đó từ file thực thi hoặc từ backing storage vào RAM.

Nếu RAM còn trống, kernel có thể cấp một frame vật lý mới cho trang cần nạp. Nếu RAM không còn đủ, kernel phải chọn một victim page để thay thế. Nếu victim page đã bị sửa đổi, kernel cần ghi nó ra swap hoặc backing store trước khi tái sử dụng frame. Nếu victim page chưa bị sửa đổi, kernel có thể bỏ nó đi vì có thể nạp lại từ file gốc khi cần. Cơ chế này giúp hệ thống sử dụng RAM hiệu quả hơn nhưng cũng tạo ra chi phí khi page fault xảy ra quá nhiều.

Page fault cũng có thể là lỗi thật. Nếu process truy cập địa chỉ không thuộc vùng nhớ hợp lệ, ghi vào vùng chỉ đọc hoặc truy cập kernel space từ user space, kernel sẽ không thể xử lý bằng cách nạp page. Khi đó, kernel thường gửi signal `SIGSEGV` đến process. Đây là nguyên nhân phổ biến của lỗi segmentation fault trong chương trình C khi dùng con trỏ sai, truy cập mảng vượt giới hạn hoặc dùng vùng nhớ đã giải phóng.

6.4 Memory mapping và chia sẻ dữ liệu

Memory mapping là cơ chế ánh xạ file hoặc vùng nhớ vào không gian địa chỉ ảo của process. Hàm `mmap` cho phép một file được xem như một vùng nhớ. Khi process đọc hoặc ghi vào vùng này, kernel sẽ xử lý việc đồng bộ với file phía sau. Cách làm này có thể hiệu quả hơn việc gọi `read` và `write` nhiều lần, đặc biệt với file lớn hoặc dữ liệu cần truy cập ngẫu nhiên.

Memory mapping cũng là nền tảng cho shared library. Khi nhiều process cùng sử dụng một thư viện động như glibc, phần code chỉ đọc của thư viện có thể được ánh xạ chung vào nhiều process. Mỗi process nhìn thấy thư viện trong không gian địa chỉ riêng của nó, nhưng thực tế các trang vật lý chứa code có thể được chia sẻ. Điều này tiết kiệm RAM đáng kể trên hệ thống Linux.

Trong IPC, `mmap` còn được dùng để tạo shared memory. Nhiều process có thể ánh xạ cùng một vùng nhớ vật lý vào không gian địa chỉ của mình. Khi một process ghi dữ liệu, process khác có thể đọc được mà không cần copy qua kernel nhiều lần. Tuy nhiên, shared memory không tự giải quyết vấn đề đồng bộ. Nếu nhiều process cùng ghi vào một vùng dữ liệu, chương trình cần semaphore, mutex liên tiến trình hoặc cơ chế khóa khác để tránh race condition.

6.5 Ý nghĩa đối với Embedded Linux

Trong Embedded Linux, virtual memory giúp hệ thống ổn định hơn vì lỗi của một ứng dụng user space thường không phá hỏng toàn bộ kernel. Tuy nhiên, tài nguyên của thiết bị nhúng thường hạn chế, nên lập trình viên phải hiểu rõ heap, stack, `mmap` và shared library để tránh lãng phí RAM. Một memory leak nhỏ trên PC có thể không đáng kể, nhưng trên thiết bị chạy liên tục nhiều ngày hoặc nhiều tháng, nó có thể làm service bị kill hoặc thiết bị mất ổn định.

Khi debug ứng dụng nhúng, hiểu virtual memory giúp phân tích các lỗi như segmentation fault, lỗi

không load được shared library, lỗi thiếu quyền truy cập vùng nhớ hoặc lỗi mmap thiết bị. Các công cụ như `pmap`, `/proc/[pid]/maps`, `strace`, `gdb` và log kernel giúp quan sát cách process sử dụng bộ nhớ. Đây là nền tảng quan trọng trước khi đi sâu vào driver, DMA buffer hoặc memory mapped I/O.

7 Process trong Linux

7.1 Khái niệm process và tài nguyên của process

Process là một chương trình đang được thực thi. Khi một file executable được chạy, kernel tạo ra một process với không gian địa chỉ riêng, PID riêng, trạng thái CPU, bảng file descriptor, thông tin quyền hạn, biến môi trường và nhiều cấu trúc quản lý khác. Process không chỉ là mã lệnh; nó là một thực thể sống trong hệ điều hành, được scheduler cấp CPU và được kernel quản lý tài nguyên.

Mỗi process có PID, tức Process ID. Ngoài ra, process thường có PPID để chỉ process cha. Quan hệ cha con này rất quan trọng trong Linux vì nhiều process được tạo ra bằng cách fork từ một process khác. Khi quan sát hệ thống bằng các lệnh như `ps`, `top` hoặc `pstree`, có thể thấy cây process phản ánh cách các chương trình được khởi động và quản lý.

Process có không gian địa chỉ riêng, vì vậy biến toàn cục hoặc heap của process này không thể bị process khác truy cập trực tiếp. Điều này tạo sự cô lập và bảo vệ hệ thống. Tuy nhiên, process vẫn có thể giao tiếp với bên ngoài thông qua file descriptor, socket, pipe, shared memory hoặc các cơ chế IPC khác. Cách Linux biểu diễn file, thiết bị, socket và pipe bằng file descriptor giúp process tương tác với nhiều loại tài nguyên theo một mô hình tương đối thống nhất.

7.2 Trạng thái của process

Trong vòng đời của mình, process có thể đi qua nhiều trạng thái. Khi process đang được CPU thực thi, nó ở trạng thái running. Khi process sẵn sàng chạy nhưng chưa được scheduler chọn, nó ở trạng thái runnable. Khi process chờ I/O, chờ signal, chờ lock hoặc chờ dữ liệu từ một process khác, nó ở trạng thái sleeping hoặc blocked. Khi process bị dừng bởi debugger hoặc signal, nó có thể ở trạng thái stopped.

Một trạng thái đặc biệt là zombie. Zombie process là process đã kết thúc thực thi nhưng entry của nó vẫn còn trong bảng process vì process cha chưa gọi `wait` hoặc `waitpid` để thu hồi exit status. Zombie không còn chạy và không chiếm CPU, nhưng vẫn chiếm một slot trong bảng process. Nếu chương trình cha tạo nhiều process con nhưng không thu hồi đúng cách, hệ thống có thể tích lũy nhiều zombie.

Một trường hợp khác là orphan process. Orphan process xuất hiện khi process cha kết thúc trước process con. Khi đó, process con được nhận nuôi bởi process init, thường là systemd hoặc init tương ứng. Orphan không nhất thiết là lỗi, nhưng trong thiết kế daemon hoặc service nhúng, cần hiểu rõ quan hệ cha con để tránh tạo process ngoài kiểm soát.

7.3 Tạo process bằng fork

Trong Linux, **fork** là system call dùng để tạo process con. Sau khi **fork** thành công, hệ thống có hai process gần như giống nhau: process cha và process con. Cả hai tiếp tục chạy từ ngay sau lời gọi **fork**, nhưng giá trị trả về khác nhau. Trong process cha, **fork** trả về PID của process con. Trong process con, **fork** trả về 0. Nếu lỗi, **fork** trả về -1.

Cơ chế **fork** ban đầu có vẻ tốn kém vì phải nhân bản process, nhưng Linux tối ưu bằng copy-on-write. Sau khi **fork**, cha và con có thể tạm thời chia sẻ các trang vật lý chỉ đọc. Chỉ khi một trong hai process ghi vào trang đó, kernel mới copy trang riêng cho process ghi. Cơ chế này làm việc tạo process hiệu quả hơn, đặc biệt khi process con sẽ gọi **exec** ngay sau đó.

Trong thực tế, mô hình phổ biến là **fork** rồi **exec**. Process cha tạo process con, process con gọi **exec** để thay thế image hiện tại bằng chương trình khác, còn process cha gọi **wait** để chờ kết thúc. Đây là cách shell chạy một lệnh, và cũng là mô hình nền tảng cho nhiều daemon hoặc service manager.

7.4 exec và wait

Họ hàm **exec** không tạo process mới mà thay thế chương trình đang chạy trong process hiện tại bằng một chương trình khác. PID của process không đổi, nhưng code, data, heap, stack và memory layout được thay bằng image mới. Điều này giải thích tại sao shell phải **fork** trước rồi process con mới **exec**; nếu shell gọi **exec** trực tiếp, chính shell sẽ bị thay thế bởi chương trình mới.

wait và **waitpid** cho phép process cha chờ process con kết thúc và thu hồi trạng thái thoát. Nếu process cha không gọi các hàm này, process con sau khi kết thúc có thể trở thành zombie. Trong chương trình tạo nhiều process con, **waitpid** thường linh hoạt hơn vì có thể chờ một PID cụ thể hoặc kiểm tra trạng thái không blocking.

Trong Embedded Linux, quản lý đúng **fork**, **exec** và **wait** rất quan trọng. Nhiều hệ thống dùng script hoặc daemon để restart service, gọi tool cấu hình, chạy chương trình phụ hoặc thu thập log. Nếu không kiểm soát process con, hệ thống có thể bị leak zombie, chạy trùng service hoặc không phát hiện được lỗi của chương trình con.

7.5 Daemon và service trong hệ thống nhúng

Daemon là process chạy nền, thường không gắn trực tiếp với terminal. Trong hệ thống nhúng, daemon có thể quản lý Wi-Fi, mạng, LED, nút nhấn, watchdog, telemetry hoặc giao tiếp cloud. Daemon cần xử lý signal đúng cách, ghi log rõ ràng, đóng file descriptor khi thoát và tránh leak tài nguyên.

Khi daemon được quản lý bởi systemd, procd hoặc init script, vòng đời của process còn liên quan đến service manager. Service manager có thể khởi động, dừng, restart và giám sát process. Điều này liên hệ trực tiếp với kiến trúc process: một service thực tế không chỉ là một file thực thi, mà là một process có vòng đời, trạng thái, log và chính sách restart.

8 Thread trong Linux

8.1 Khái niệm thread

Thread là đơn vị thực thi nhỏ nhất được scheduler điều phối. Một process có thể có một hoặc nhiều thread. Các thread trong cùng process chia sẻ không gian địa chỉ, heap, biến toàn cục, vùng mmap và file descriptor. Tuy nhiên, mỗi thread có stack riêng, program counter riêng, register context riêng và trạng thái thực thi riêng.

So với process, thread nhẹ hơn vì không cần tạo không gian địa chỉ hoàn toàn mới. Việc trao đổi dữ liệu giữa các thread cũng đơn giản hơn vì chúng có thể truy cập chung biến trong cùng process. Tuy nhiên, chính sự chia sẻ này làm lập trình thread dễ phát sinh lỗi đồng bộ. Nếu hai thread cùng ghi vào một biến mà không có khóa, kết quả có thể phụ thuộc vào thứ tự chạy không xác định của scheduler.

Trong Linux, lập trình thread C thường dùng POSIX Thread, hay pthread. Hàm `pthread_create` tạo thread mới, `pthread_join` chờ thread kết thúc, `pthread_exit` kết thúc thread hiện tại. Khi biên dịch chương trình pthread, thường cần liên kết với tùy chọn `-pthread` để compiler và linker cấu hình đúng thư viện thread.

8.2 So sánh process và thread

Process có mức cô lập cao hơn thread. Mỗi process có không gian địa chỉ riêng, nên lỗi ghi nhầm bộ nhớ thường chỉ ảnh hưởng process đó. Giao tiếp giữa process cần IPC như pipe, socket hoặc shared memory. Ngược lại, thread chia sẻ bộ nhớ trong cùng process nên trao đổi dữ liệu nhanh hơn, nhưng lỗi của một thread có thể làm hỏng toàn bộ process.

Tạo process thường tốn tài nguyên hơn tạo thread, dù Linux đã tối ưu bằng copy-on-write. Thread phù hợp khi các tác vụ cần chia sẻ dữ liệu thường xuyên, ví dụ một thread nhận dữ liệu mạng, một thread xử lý dữ liệu, một thread ghi log. Process phù hợp khi cần cô lập mạnh hơn, ví dụ tách một thành phần có nguy cơ crash hoặc chạy các chương trình độc lập.

Trong hệ thống nhúng, lựa chọn process hay thread phụ thuộc vào bài toán. Nếu thiết bị có ít RAM và cần chia sẻ buffer lớn, thread có thể hiệu quả hơn. Nếu hệ thống cần độ ổn định cao và muốn lỗi của một module không làm chết toàn bộ service, nhiều process có thể an toàn hơn. Thiết kế tốt thường kết hợp cả hai mô hình.

8.3 Race condition và critical section

Race condition xảy ra khi kết quả chương trình phụ thuộc vào thứ tự thực thi giữa các thread. Ví dụ, hai thread cùng tăng một biến đếm toàn cục. Lệnh tăng biến trong C có thể gồm nhiều bước ở mức máy: đọc giá trị, cộng thêm một, ghi lại. Nếu hai thread xen kẽ nhau giữa các bước này, một lần cập nhật có thể bị mất.

Critical section là vùng code truy cập tài nguyên dùng chung và cần được bảo vệ. Tài nguyên dùng chung có thể là biến toàn cục, queue, linked list, file descriptor, socket hoặc buffer. Khi một thread

đang ở trong critical section, các thread khác không được phép vào cùng vùng bảo vệ đó nếu có nguy cơ làm sai dữ liệu.

Mutex là cơ chế phổ biến để bảo vệ critical section. Thread gọi `pthread_mutex_lock` trước khi truy cập dữ liệu chung và gọi `pthread_mutex_unlock` sau khi xong. Nếu mutex đang bị giữ bởi thread khác, thread hiện tại sẽ chờ. Cách dùng mutex phải cẩn thận để tránh quên unlock, lock sai thứ tự hoặc giữ lock quá lâu làm giảm hiệu năng.

8.4 Deadlock, condition variable và semaphore

Deadlock xảy ra khi các thread chờ nhau theo vòng tròn và không thread nào tiếp tục được. Ví dụ, thread A giữ mutex 1 và chờ mutex 2, trong khi thread B giữ mutex 2 và chờ mutex 1. Để tránh deadlock, chương trình cần quy định thứ tự lock rõ ràng, giảm số lock lồng nhau và đảm bảo mọi đường thoát đều unlock đúng cách.

Condition variable được dùng khi thread cần chờ một điều kiện logic thay vì chỉ chờ mutex. Ví dụ, một thread producer đưa dữ liệu vào queue, thread consumer chờ queue có dữ liệu. Consumer có thể ngủ trên condition variable và được đánh thức khi producer thêm dữ liệu. Cơ chế này hiệu quả hơn việc liên tục polling.

Semaphore có thể dùng để đếm tài nguyên hoặc đồng bộ giữa thread/process. Nếu semaphore có giá trị lớn hơn 0, thread có thể đi tiếp và giảm giá trị. Nếu giá trị bằng 0, thread phải chờ. Semaphore phù hợp với bài toán giới hạn số tài nguyên đồng thời, hoặc báo hiệu số lượng phần tử trong một buffer.

8.5 Ứng dụng thread trong Embedded Linux

Trong ứng dụng nhúng, thread thường được dùng để tổ chức các tác vụ song song. Một thiết bị gateway có thể có thread đọc socket, thread xử lý dữ liệu, thread giao tiếp UART và thread ghi log. Một AP có thể có thread nhận lệnh cấu hình, thread theo dõi trạng thái client và thread gửi telemetry về server.

Tuy nhiên, dùng nhiều thread không tự động làm chương trình tốt hơn. Nếu thiết kế đồng bộ kém, hệ thống có thể sinh race condition, deadlock hoặc lỗi timing khó tái hiện. Vì vậy, khi dùng thread trong Embedded Linux, cần xác định rõ dữ liệu nào dùng chung, thread nào sở hữu tài nguyên nào, khi nào cần lock và khi nào nên dùng message queue thay vì chia sẻ biến trực tiếp.

9 Interprocess Communication

9.1 Nhu cầu giao tiếp liên tiến trình

Mỗi process Linux có không gian địa chỉ riêng, vì vậy process này không thể tự ý đọc ghi biến của process khác. Sự cô lập này giúp hệ thống an toàn hơn nhưng đặt ra nhu cầu giao tiếp liên tiến trình, gọi là IPC. IPC cho phép các process trao đổi dữ liệu, gửi tín hiệu điều khiển hoặc đồng bộ thứ tự thực thi.

Trong hệ thống nhúng thực tế, IPC xuất hiện ở rất nhiều nơi. Một web backend cần gửi yêu cầu đến daemon cấu hình Wi-Fi. Một process đọc cảm biến cần gửi dữ liệu cho process xử lý. Một service quản lý cần nhận trạng thái từ nhiều process con. Một ứng dụng cần gửi log hoặc event đến process giám sát. Vì vậy, hiểu IPC là điều kiện cần để thiết kế hệ thống Linux nhiều process.

9.2 Pipe và FIFO

Pipe là cơ chế IPC đơn giản, thường dùng để truyền dữ liệu một chiều giữa các process có quan hệ gần, đặc biệt là cha con. Trong shell, dấu `|` tạo pipeline giữa stdout của process trước và stdin của process sau. Pipe hoạt động như một buffer trong kernel. Process ghi dữ liệu vào một đầu, process khác đọc dữ liệu từ đầu còn lại.

Pipe thông thường không có tên trong filesystem, vì vậy nó phù hợp cho các process được tạo cùng thời điểm hoặc có quan hệ kế thừa file descriptor. FIFO, hay named pipe, mở rộng pipe bằng cách có một tên trong filesystem. Nhờ đó, hai process không có quan hệ cha con vẫn có thể mở cùng FIFO để trao đổi dữ liệu.

Pipe và FIFO đơn giản, dễ dùng, phù hợp với dòng dữ liệu tuần tự. Tuy nhiên, chúng không phù hợp nếu cần truy cập ngẫu nhiên, chia sẻ buffer lớn hoặc gửi thông điệp có cấu trúc phức tạp. Khi dữ liệu lớn hoặc cần nhiều client, socket hoặc shared memory có thể phù hợp hơn.

9.3 Message queue

Message queue cho phép process gửi và nhận các thông điệp rời rạc. Khác với pipe là dòng byte liên tục, message queue giữ ranh giới từng message. Điều này thuận tiện khi dữ liệu có cấu trúc như command, event, trạng thái hoặc gói cấu hình.

Trong POSIX message queue, mỗi message có kích thước và có thể có độ ưu tiên. Process nhận có thể chờ đến khi có message mới. Cơ chế này phù hợp cho mô hình producer-consumer hoặc điều khiển daemon bằng các lệnh rời rạc. Tuy nhiên, message queue có giới hạn về kích thước message và số lượng message, nên không nên dùng để truyền dữ liệu quá lớn.

9.4 Shared memory

Shared memory là cơ chế IPC có hiệu năng cao vì nhiều process cùng ánh xạ một vùng nhớ chung. Sau khi mapping hoàn tất, dữ liệu không cần copy qua kernel trong mỗi lần trao đổi. Điều này rất hữu ích với buffer lớn như dữ liệu camera, audio, packet buffer hoặc bảng trạng thái lớn.

Điểm yếu của shared memory là nó chỉ cung cấp vùng nhớ chung, không cung cấp đồng bộ. Nếu một process đang ghi và process khác đọc cùng lúc, dữ liệu có thể không nhất quán. Vì vậy, shared memory thường phải đi kèm semaphore, mutex liên tiến trình hoặc cơ chế lock tự thiết kế. Khi process crash, chương trình cũng cần xử lý việc dọn dẹp vùng shared memory để tránh rò rỉ tài nguyên hệ thống.

9.5 Semaphore và đồng bộ process

Semaphore dùng để đồng bộ hoặc giới hạn truy cập tài nguyên. Trong IPC, semaphore có thể bảo vệ shared memory, báo hiệu dữ liệu mới hoặc điều phối thứ tự chạy giữa các process. Một process có thể giảm semaphore trước khi vào vùng critical section và tăng semaphore khi rời khỏi vùng đó. Process khác nếu thấy semaphore chưa sẵn sàng sẽ phải chờ.

Semaphore rất hữu ích nhưng cũng có rủi ro. Nếu process giữ semaphore rồi crash, các process khác có thể bị chờ vô hạn nếu không có cơ chế phục hồi. Vì vậy, trong hệ thống nhúng cần chạy lâu dài, thiết kế đồng bộ phải tính đến timeout, cleanup và khởi tạo lại trạng thái sau lỗi.

9.6 Socket như một cơ chế IPC

Socket không chỉ dùng cho giao tiếp qua mạng mà còn dùng cho IPC trên cùng máy. Unix domain socket cho phép các process trên cùng hệ thống trao đổi dữ liệu qua một endpoint trong filesystem. So với TCP socket, Unix domain socket không đi qua tầng IP nên hiệu quả hơn cho giao tiếp nội bộ.

Trong Embedded Linux, Unix domain socket thường được dùng giữa web server, daemon quản lý và service hệ thống. TCP/UDP socket được dùng khi cần giao tiếp qua mạng LAN, WAN hoặc cloud. Socket có ưu điểm là mô hình client-server rõ ràng, hỗ trợ nhiều kết nối và dễ tích hợp với event loop bằng `select`, `poll` hoặc `epoll`.

9.7 Lựa chọn IPC trong thiết kế nhúng

Không có cơ chế IPC nào tốt nhất cho mọi bài toán. Pipe phù hợp cho dòng dữ liệu đơn giản. FIFO phù hợp khi cần tên cố định trong filesystem. Message queue phù hợp cho command/event có cấu trúc. Shared memory phù hợp cho dữ liệu lớn và yêu cầu tốc độ cao. Semaphore phù hợp cho đồng bộ. Socket phù hợp cho mô hình client-server hoặc giao tiếp mạng.

Khi thiết kế hệ thống nhúng, cần cân nhắc kích thước dữ liệu, tần suất truyền, số process tham gia, yêu cầu realtime, khả năng phục hồi lỗi và độ phức tạp triển khai. Một thiết kế tốt thường kết hợp nhiều cơ chế IPC, ví dụ dùng socket để nhận lệnh điều khiển, message queue để phát event nội bộ và shared memory để truyền buffer lớn.

10 Signal trong Linux

10.1 Khái niệm signal

Signal là cơ chế thông báo bất đồng bộ trong Linux. Nó cho phép kernel hoặc một process gửi thông báo đến process khác khi có sự kiện xảy ra. Signal không mang nhiều dữ liệu như message queue, nhưng rất hữu ích để báo các sự kiện như yêu cầu dừng chương trình, lỗi truy cập bộ nhớ, timer hết hạn hoặc process con kết thúc.

Signal có thể được tạo ra từ nhiều nguồn. Người dùng nhấn `Ctrl+C` trong terminal thường tạo `SIGINT`. Lệnh `kill` có thể gửi signal đến một process theo PID. Kernel gửi `SIGSEGV` khi process

truy cập bộ nhớ sai. Kernel gửi **SIGCHLD** cho process cha khi process con kết thúc. Timer có thể tạo **SIGALRM** khi đến thời điểm đã đặt.

10.2 Một số signal thường gặp

SIGINT thường được dùng để ngắt chương trình từ bàn phím. **SIGTERM** là yêu cầu kết thúc mềm, cho phép chương trình dọn dẹp trước khi thoát. **SIGKILL** buộc process kết thúc ngay và không thể bị bắt hoặc bỏ qua. **SIGSEGV** báo lỗi truy cập bộ nhớ không hợp lệ. **SIGCHLD** báo cho process cha biết process con đã kết thúc hoặc thay đổi trạng thái. **SIGALRM** được dùng với timer.

Sự khác biệt giữa **SIGTERM** và **SIGKILL** rất quan trọng trong vận hành hệ thống. Khi dùng một daemon, nên gửi **SIGTERM** trước để nó có cơ hội đóng socket, ghi log, flush dữ liệu và giải phóng tài nguyên. Chỉ khi daemon không phản hồi mới dùng **SIGKILL**. Nếu thường xuyên dùng **SIGKILL**, hệ thống có thể để lại file tạm, lock file, shared memory hoặc trạng thái thiết bị chưa được dọn dẹp.

10.3 Xử lý signal bằng signal và sigaction

Chương trình có thể đăng ký handler để xử lý signal. Hàm **signal** đơn giản nhưng có một số khác biệt hành vi giữa các hệ thống. Hàm **sigaction** được khuyến nghị hơn vì cho phép cấu hình rõ ràng handler, mask và flags. Với **sigaction**, chương trình có thể xác định signal nào bị block trong lúc handler chạy và có thể nhận thêm thông tin về signal trong một số chế độ.

Khi viết signal handler, cần cẩn thận vì handler có thể chạy bất kỳ lúc nào, chen vào luồng thực thi bình thường. Không phải hàm nào cũng an toàn để gọi trong signal handler. Trong chương trình thực tế, handler thường chỉ đặt một biến cờ kiểu **volatile sig_atomic_t**, sau đó vòng lặp chính kiểm tra cờ này và thực hiện dọn dẹp ở ngữ cảnh an toàn hơn.

10.4 Signal và quản lý process con

SIGCHLD có vai trò quan trọng khi một process tạo process con. Khi process con kết thúc, kernel gửi **SIGCHLD** cho process cha. Nếu process cha bỏ qua việc thu hồi exit status, process con có thể trở thành zombie. Vì vậy, nhiều daemon đăng ký handler cho **SIGCHLD** hoặc dùng vòng lặp **waitpid** để thu hồi process con.

Trong một server hoặc service nhúng, nếu mỗi request hoặc mỗi tác vụ phụ tạo process con, việc xử lý **SIGCHLD** càng quan trọng. Không quản lý đúng signal này có thể làm hệ thống tích lũy zombie theo thời gian. Đây là ví dụ cho thấy signal liên hệ trực tiếp với phần process đã học.

10.5 Signal trong daemon Embedded Linux

Một daemon chạy trên Embedded Linux nên xử lý signal một cách có chủ đích. Khi nhận **SIGTERM**, daemon nên dừng nhận request mới, đóng socket, lưu trạng thái cần thiết, giải phóng shared memory, unlock tài nguyên và thoát với mã trạng thái rõ ràng. Khi nhận signal reload cấu hình, ví

dụ **SIGHUP** trong một số thiết kế, daemon có thể đọc lại file cấu hình mà không cần restart toàn bộ process.

Trong hệ thống router hoặc AP, signal thường xuất hiện khi service Wi-Fi restart, network reload hoặc hệ thống shutdown. Nếu daemon quản lý Wi-Fi không xử lý signal tốt, nó có thể để lại interface ở trạng thái không nhất quán hoặc không đóng kết nối quản lý. Vì vậy, signal là một phần nhỏ về mặt API nhưng rất quan trọng về độ ổn định vận hành.

11 Linux Device Driver và giao tiếp với phần cứng

11.1 Vai trò của device driver

Device driver là lớp mã kernel hiểu cả giao diện của Linux lẫn cách vận hành của một thiết bị cụ thể. Phía trên driver, kernel và ứng dụng muốn thao tác theo các khái niệm chung như đọc dữ liệu, gửi packet, cấu hình mode hoặc chờ event. Phía dưới driver là thanh ghi, vùng nhớ I/O, interrupt, DMA, bus và firmware của phần cứng. Driver chịu trách nhiệm chuyển đổi giữa hai thế giới đó.

Một nguyên tắc thiết kế quan trọng là tách mechanism khỏi policy. Driver nên cung cấp khả năng truy cập phần cứng và thực thi các thao tác hợp lệ, còn quyết định sử dụng thiết bị theo cách nào nên đặt ở tầng user space hoặc subsystem cao hơn. Ví dụ, Wi-Fi driver cung cấp cơ chế đặt channel, tạo interface, truyền nhận frame và báo event; policy chọn SSID, lịch restart radio hoặc quyền người dùng thay đổi cấu hình thuộc về daemon và hệ thống quản lý.

Sự phân tách này làm driver tái sử dụng được trong nhiều sản phẩm. Cùng một driver có thể phục vụ router gia đình, gateway công nghiệp hoặc thiết bị test, trong khi mỗi sản phẩm áp dụng policy riêng ở user space. Nếu driver hard-code quá nhiều policy, việc bảo trì và tích hợp với OpenWrt hoặc framework khác sẽ khó hơn.

11.2 Module kernel và vòng đời driver

Driver có thể được build trực tiếp vào kernel hoặc build thành loadable kernel module. Module cho phép nạp và gỡ mã kernel khi hệ thống đang chạy, thuận tiện trong phát triển và debug. Một module thường có hàm khởi tạo để đăng ký driver hoặc tài nguyên với kernel, và hàm cleanup để hủy đăng ký, giải phóng bộ nhớ, interrupt và các tài nguyên đã lấy.

Vòng đời driver cần có tính đối xứng. Nếu bước khởi tạo đã request một vùng I/O, đăng ký interrupt, tạo device node hoặc cấp phát buffer, đường xử lý lỗi và hàm cleanup phải giải phóng đúng theo thứ tự ngược lại. Nếu bỏ sót, việc reload module có thể thất bại, tài nguyên kernel bị leak hoặc thiết bị còn ở trạng thái không xác định. Đây là nguyên nhân phổ biến khiến driver chạy được lần đầu nhưng lỗi sau vài lần load/unload.

Module còn có thể nhận parameter để hỗ trợ debug hoặc chọn cấu hình khi nạp. Tuy nhiên, cấu hình phụ thuộc board thường nên được mô tả bằng Device Tree, ACPI hoặc dữ liệu từ bus thay vì hard-code trong source. Trên ARM Embedded Linux, hàm **probe** thường nhận tài nguyên đã được kernel phân tích từ Device Tree và tiến hành khởi tạo thiết bị.

11.3 Char, block và network driver

Linux thường phân loại driver theo cách dữ liệu được cung cấp cho hệ thống. Character driver biểu diễn dữ liệu như một dòng byte và thường được truy cập thông qua device node trong `/dev`.

UART, GPIO character interface hoặc thiết bị đo lường đơn giản có thể dùng mô hình này. Block driver cung cấp các block dữ liệu có thể truy cập ngẫu nhiên và tham gia vào storage stack, ví dụ eMMC, SD hoặc disk. Network driver không chủ yếu cung cấp `read/write` qua device node mà kết nối với network stack để truyền nhận packet.

Sự phân loại quyết định subsystem và callback mà driver phải triển khai. Character driver thường đăng ký major/minor number và một tập `file_operations`. Block driver làm việc với request queue và block layer. Network driver đăng ký một network device, quản lý queue truyền nhận, thống kê và trạng thái link. Wi-Fi còn có thêm các lớp chuyên biệt như `cfg80211/mac80211` hoặc vendor interface tùy kiến trúc driver.

Đối với MediaTek AP, Ethernet và Wi-Fi nằm trong nhóm network device, nhưng hệ thống vẫn có thể cung cấp thêm char device, procfs, sysfs, debugfs hoặc ioctl để điều khiển và lấy trạng thái. Vì vậy, khi debug cần phân biệt data path truyền packet với control path cấu hình radio.

11.4 `file_operations` và đường đi của lời gọi hệ thống

Với character driver, cấu trúc `file_operations` liên kết các lời gọi từ user space với callback trong driver. Khi ứng dụng gọi `open`, VFS tìm device node và gọi hàm `open` của driver. Tương tự, `read`, `write`, `poll`, `mmap` hoặc `ioctl` được chuyển đến callback tương ứng nếu driver cung cấp.

Callback driver phải tuân theo quy ước của kernel về giá trị trả về và mã lỗi. Hàm đọc có thể trả số byte đã đọc, trả 0 khi hết dữ liệu theo ngữ nghĩa phù hợp, hoặc trả giá trị âm biểu diễn lỗi. Khi trao đổi buffer với user space, driver không nên dereference trực tiếp con trỏ user. Nó phải dùng các helper như `copy_to_user` và `copy_from_user` để kernel kiểm tra và xử lý lỗi truy cập.

Nếu chưa có dữ liệu, driver có thể đưa process gọi `read` vào wait queue thay vì busy-wait. Khi interrupt hoặc một event khác báo dữ liệu đã sẵn sàng, driver đánh thức process. Cơ chế này kết nối trực tiếp các phần đã học: system call đi vào driver, process chuyển sang sleeping, interrupt cập nhật trạng thái, wait queue đánh thức process và scheduler cho process tiếp tục.

11.5 Concurrency và bảo vệ dữ liệu trong kernel

Driver có thể được truy cập đồng thời từ nhiều nguồn: nhiều process user space, nhiều thread, interrupt handler, timer, workqueue hoặc nhiều CPU. Vì vậy, biến toàn cục hoặc cấu trúc thiết bị không thể được giả định chỉ có một luồng sử dụng. Race condition trong kernel nguy hiểm hơn user space vì có thể làm hỏng dữ liệu chung hoặc gây kernel panic.

Mutex phù hợp cho ngữ cảnh process khi code có thể ngủ. Spinlock phù hợp cho critical section rất ngắn hoặc dữ liệu được chia sẻ với interrupt context, nơi không được phép ngủ. Atomic operation phù hợp cho counter hoặc thay đổi trạng thái nhỏ. Wait queue và completion dùng để chờ event. Việc chọn sai primitive có thể dẫn đến deadlock, sleep trong atomic context hoặc giữ interrupt quá lâu.

Quy tắc thực tế là xác định rõ dữ liệu nào được chia sẻ, các context nào có thể truy cập và thứ tự lock. Driver cũng cần tránh gọi hàm có thể ngủ trong khi đang giữ spinlock hoặc ở interrupt handler. Các nguyên lý race condition, critical section và deadlock từ lập trình thread vì vậy tiếp tục áp dụng ở kernel, nhưng yêu cầu nghiêm ngặt hơn.

11.6 I/O memory, thanh ghi và memory barrier

Nhiều thiết bị nhúng được điều khiển qua các thanh ghi memory-mapped I/O. Device Tree cung cấp địa chỉ vật lý và kích thước vùng thanh ghi; driver nhận tài nguyên, request vùng tương ứng và ánh xạ nó vào không gian địa chỉ kernel bằng cơ chế như `ioremap` hoặc helper quản lý tài nguyên. Sau đó driver dùng các accessor như `readl` và `writel` thay vì dereference con trỏ thông thường.

Các accessor tồn tại vì truy cập I/O không hoàn toàn giống truy cập RAM. Kiến trúc CPU có thể khác nhau về endianness, thứ tự thực thi và cách ánh xạ bus. Compiler hoặc CPU có thể reorder thao tác bộ nhớ nếu không có ràng buộc phù hợp. Driver phải dùng API I/O và memory barrier đúng để đảm bảo thiết bị nhìn thấy các thao tác theo thứ tự yêu cầu.

Địa chỉ load kernel hay địa chỉ thanh ghi trong tài liệu SoC là địa chỉ vật lý, trong khi con trỏ driver thường là địa chỉ ảo kernel sau mapping. Nhầm lẫn hai loại địa chỉ là lỗi phổ biến khi mới làm Embedded Linux. Driver cũng phải request tài nguyên trước khi sử dụng để tránh hai driver cùng điều khiển một vùng phần cứng.

11.7 Interrupt và deferred work

Thiết bị sử dụng interrupt để báo sự kiện như nhận xong packet, hoàn tất DMA, có dữ liệu UART hoặc phát hiện lỗi. Driver đăng ký interrupt handler với kernel. Handler cần xác định interrupt có thuộc thiết bị của mình không, xác nhận hoặc clear nguồn interrupt và lưu trạng thái tối thiểu cần thiết.

Interrupt handler không được thực hiện công việc dài hoặc gọi hàm có thể ngủ. Phần xử lý nặng thường được chuyển sang threaded interrupt, workqueue, tasklet trong các thiết kế cũ, NAPI đối với network driver hoặc một cơ chế deferred work phù hợp. Với network driver, NAPI giúp chuyển từ xử lý từng packet bằng interrupt sang polling có kiểm soát khi lưu lượng cao, giảm interrupt storm và tăng throughput.

Thiết kế interrupt sai có thể gây treo hệ thống hoặc tiêu tốn CPU. Nếu không clear nguồn interrupt, CPU có thể liên tục vào handler. Nếu khóa dùng giữa handler và process context không đúng, deadlock có thể xảy ra. Nếu driver đánh thức process trước khi buffer hoàn tất, user space có thể nhận dữ liệu không nhất quán.

11.8 DMA và quản lý buffer

Thiết bị tốc độ cao như Ethernet, Wi-Fi, USB hoặc storage thường dùng DMA để truyền dữ liệu giữa thiết bị và RAM mà không yêu cầu CPU copy từng byte. Driver chuẩn bị descriptor và buffer, ánh xạ buffer cho DMA, thông báo thiết bị bắt đầu, sau đó nhận interrupt hoặc poll trạng thái khi hoàn tất.

DMA đặt ra vấn đề về địa chỉ và cache coherency. Địa chỉ CPU dùng không nhất thiết là địa chỉ thiết bị nhìn thấy. Driver phải dùng DMA API để lấy DMA address phù hợp và đồng bộ cache theo đúng hướng truyền. Không nên chuyển trực tiếp địa chỉ ảo từ `kmalloc` cho phần cứng rồi giả định nó luôn hoạt động trên mọi SoC.

Trong data path của MediaTek AP, packet có thể đi qua nhiều queue và DMA ring giữa CPU với Wi-Fi/Ethernet hardware. Khi throughput thấp hoặc mất packet, ngoài cấu hình SoftAP còn phải kiểm tra descriptor, interrupt, queue stop/wake, DMA mapping và cache coherency. Đây là lớp thấp hơn so với `iwpriv` hoặc daemon cấu hình nhưng có ảnh hưởng trực tiếp đến hiệu năng.

11.9 Linux device model, bus, device và driver

Linux device model biểu diễn quan hệ giữa bus, device và driver. Một bus hoặc subsystem phát hiện device; driver khai báo bằng định danh hoặc chuỗi `compatible`; kernel thực hiện matching và gọi `probe` khi tìm thấy cặp phù hợp. Khi device bị gỡ hoặc driver unload, callback `remove` thực hiện cleanup.

Kobject, sysfs và uevent giúp biểu diễn cấu trúc thiết bị ra user space. Thư mục dưới `/sys` cho phép quan sát driver nào đang bind, thuộc bus nào và có các attribute gì. Hotplug event cho phép user space phản ứng khi thiết bị xuất hiện. Trên board ARM không tự mô tả phần cứng, Device Tree cung cấp các node để platform bus tạo device và matching với driver.

Khi một peripheral không hoạt động, kiểm tra `/sys`, log `probe`, thuộc tính `compatible`, clock, reset, regulator, pinctrl và interrupt thường hiệu quả hơn việc chỉ nhìn device node. Device node không xuất hiện có thể là hậu quả của driver chưa probe, không phải nguyên nhân ban đầu.

11.10 Debug driver và liên hệ MediaTek AP

Debug kernel driver cần nhiều lớp quan sát. Log từ `dmesg` giúp xác định module load, probe, firmware load và lỗi tài nguyên. Sysfs cho biết device-driver binding. Procfs, debugfs hoặc công cụ vendor có thể cung cấp counter. Dynamic debug, ftrace, perf và crash dump hỗ trợ phân tích sâu hơn. Với lỗi nghiêm trọng, kernel oops và stack trace cho biết context, function và module liên quan.

Trên MediaTek AP, control path có thể đi từ UCI hoặc web UI đến daemon, script, lệnh vendor hoặc ioctl rồi xuống Wi-Fi driver. Data path đi từ network stack qua driver, queue, DMA và radio hardware. Tách hai đường này giúp khoanh vùng lỗi: đổi SSID không có hiệu lực thường thuộc control path; throughput thấp, TX timeout hoặc mất packet thường liên quan data path, interrupt, DMA, queue hoặc firmware.

Các tài liệu device driver dù mô tả một phiên bản kernel cũ vẫn cung cấp các nguyên lý bền vững: driver nên hạn chế policy, quản lý vòng đời tài nguyên đối xứng, xử lý concurrency rõ ràng, dùng API kernel thay vì truy cập phần cứng tùy tiện và giữ interrupt handler ngắn. Khi áp dụng thực tế cần đối chiếu API với kernel tree và BSP đang dùng vì tên hàm, subsystem và cơ chế nội bộ có thể đã thay đổi.

12 Git và quy trình làm việc nhóm

Git là công cụ quản lý phiên bản phân tán, giúp theo dõi lịch sử thay đổi mã nguồn và hỗ trợ làm việc nhóm. Trong dự án Embedded Linux, Git không chỉ quản lý source code ứng dụng mà còn quản lý script build, cấu hình board, file Device Tree, tài liệu kỹ thuật và đôi khi cả các patch dành cho kernel hoặc bootloader.

Quy trình làm việc với Git thường bắt đầu từ việc tạo nhánh mới cho một tính năng hoặc một lỗi cần sửa. Lập trình viên thay đổi mã nguồn, dùng `git add` để đưa thay đổi vào staging area và `git commit` để ghi lại một snapshot có ý nghĩa. Sau đó, nhánh được đẩy lên remote repository bằng `git push`. Trong môi trường GitLab, lập trình viên thường tạo Merge Request để yêu cầu review trước khi gộp vào nhánh chính. CI/CD pipeline có thể tự động build và chạy test để phát hiện lỗi sớm.

Việc không commit trực tiếp lên nhánh chính giúp giảm rủi ro làm hỏng codebase chung. Các thao tác như `git fetch`, `git pull`, `git rebase`, `git stash`, `git cherry-pick`, `git revert` và `git reset` hỗ trợ xử lý nhiều tình huống khác nhau trong quá trình phát triển. Tuy nhiên, các lệnh thay đổi lịch sử như `reset` hoặc `push -force` cần được dùng cẩn trọng, đặc biệt khi nhánh đã được chia sẻ với người khác.

13 Socket Programming bằng C trên Linux

13.1 Vai trò của socket trong Linux Programming

Socket programming là một nội dung quan trọng trong Linux Programming vì nó cung cấp cơ chế giao tiếp giữa các tiến trình thông qua giao diện mạng hoặc ngay trong cùng một máy. Nếu pipe, FIFO hay shared memory thường được dùng cho các tiến trình cục bộ, socket mở rộng khả năng giao tiếp ra môi trường mạng, cho phép một chương trình trên thiết bị này trao đổi dữ liệu với chương trình trên thiết bị khác. Trong Embedded Linux, socket thường xuất hiện trong các ứng dụng gateway, router, thiết bị IoT, hệ thống giám sát, dịch vụ điều khiển từ xa hoặc daemon trao đổi dữ liệu với server trung tâm.

Về bản chất, socket có thể được xem như một điểm cuối của kênh truyền thông. Chương trình C thao tác với socket thông qua file descriptor, tương tự như khi thao tác với file hoặc thiết bị trong Linux. Điều này thể hiện rõ triết lý “everything is a file” của Unix/Linux: sau khi socket được tạo, lập trình viên có thể dùng các lời gọi hệ thống như `read`, `write`, `send`, `recv` hoặc `close` để trao đổi dữ liệu và giải phóng tài nguyên. Nhờ đó, socket trở thành cầu nối tự nhiên giữa kiến thức system call, process, IPC và lập trình mạng.

13.2 TCP socket và UDP socket

Trong lập trình mạng C, hai kiểu socket thường gặp nhất là TCP socket và UDP socket. TCP là giao thức hướng kết nối, đảm bảo dữ liệu được truyền theo thứ tự, có kiểm soát lỗi và có cơ chế truyền lại khi mất gói. Vì vậy, TCP phù hợp với các ứng dụng cần độ tin cậy cao như truyền file, giao tiếp client-server, cấu hình thiết bị hoặc các giao thức ứng dụng yêu cầu dữ liệu đầy đủ. Tuy nhiên, TCP có chi phí lớn hơn vì cần thiết lập kết nối, duy trì trạng thái và xử lý cơ chế kiểm soát

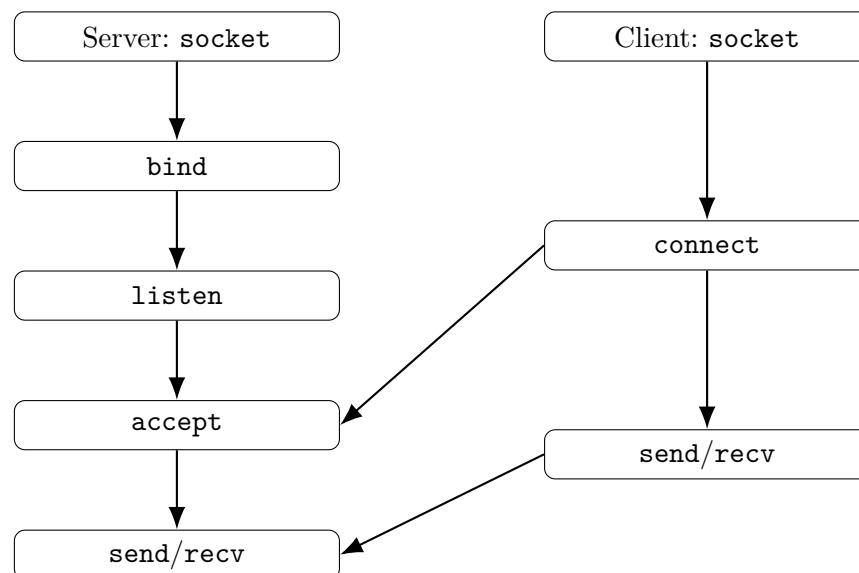
luồng.

UDP là giao thức không hướng kết nối. Khi dùng UDP, chương trình gửi datagram đến địa chỉ đích mà không cần thiết lập kết nối trước. UDP không đảm bảo gói tin đến nơi, không đảm bảo thứ tự và không tự truyền lại khi mất gói. Đổi lại, UDP đơn giản hơn, độ trễ thấp hơn và phù hợp với các tình huống như truyền dữ liệu cảm biến định kỳ, log nhẹ, broadcast/multicast trong mạng nội bộ hoặc các ứng dụng tự xây dựng cơ chế kiểm soát lỗi ở tầng trên. Trong hệ thống nhúng, việc chọn TCP hay UDP phụ thuộc vào yêu cầu về độ tin cậy, độ trễ, băng thông và tài nguyên xử lý của thiết bị.

13.3 Luồng hoạt động của chương trình server và client

Một chương trình TCP server thường bắt đầu bằng lời gọi `socket` để tạo socket endpoint. Sau đó, server dùng `bind` để gắn socket với địa chỉ IP và port cục bộ, dùng `listen` để chuyển socket sang trạng thái chờ kết nối và dùng `accept` để nhận kết nối từ client. Khi kết nối được chấp nhận, server nhận được một file descriptor mới đại diện cho phiên giao tiếp với client đó. Từ thời điểm này, server có thể dùng `send/recv` hoặc `read/write` để trao đổi dữ liệu. Khi kết thúc, server đóng socket bằng `close`.

Ở phía client, luồng hoạt động thường ngắn hơn. Client tạo socket bằng `socket`, sau đó gọi `connect` để kết nối đến IP và port của server. Nếu kết nối thành công, client trao đổi dữ liệu với server và đóng socket khi hoàn tất. Với UDP, luồng chương trình đơn giản hơn vì không cần `listen`, `accept` hay `connect` bắt buộc; chương trình thường dùng `sendto` và `recvfrom` để gửi nhận từng datagram.



13.4 Những vấn đề cần chú ý khi lập trình socket C

Khi lập trình socket bằng C, người học cần chú ý đến nhiều vấn đề thực tế. Thứ nhất là biểu diễn địa chỉ mạng. Các cấu trúc như `struct sockaddr_in`, `inet_pton` và `htons` được dùng để chuyển

đổi địa chỉ IP, port và thứ tự byte giữa host byte order và network byte order. Nếu xử lý sai byte order, chương trình có thể kết nối nhầm port hoặc không giao tiếp được.

Thứ hai là vấn đề blocking và non-blocking I/O. Theo mặc định, nhiều lời gọi socket có thể chặn chương trình cho đến khi có dữ liệu hoặc đến khi kết nối hoàn tất. Với chương trình đơn giản, blocking I/O dễ viết và dễ hiểu. Tuy nhiên, với server cần phục vụ nhiều client hoặc daemon nhúng cần xử lý đồng thời nhiều nguồn dữ liệu, lập trình viên cần cân nhắc dùng thread, `select`, `poll`, `epoll` hoặc non-blocking socket. Đây là điểm liên hệ trực tiếp giữa socket programming, thread, process và IPC.

Thứ ba là xử lý lỗi và đóng tài nguyên. Các lời gọi như `socket`, `bind`, `connect`, `send` và `recv` đều có thể thất bại. Chương trình cần kiểm tra giá trị trả về, dùng `errno` khi cần, đóng file descriptor đúng lúc và xử lý trường hợp peer đóng kết nối. Trong Embedded Linux, việc bỏ sót đóng socket hoặc tạo kết nối lặp lại không kiểm soát có thể làm cạn file descriptor, tăng bộ nhớ sử dụng và gây mất ổn định hệ thống chạy lâu dài.

14 OpenWrt trong hệ sinh thái Embedded Linux

14.1 Tổng quan về OpenWrt

OpenWrt là một hệ điều hành Linux dành cho các thiết bị mạng nhúng, đặc biệt phổ biến trên router, access point, gateway và các thiết bị có chức năng định tuyến. Khác với firmware đóng của nhiều nhà sản xuất, OpenWrt cung cấp một môi trường Linux linh hoạt, có hệ thống quản lý gói, cấu hình rõ ràng và khả năng tùy biến cao. Điều này giúp người phát triển có thể biến một thiết bị mạng nhỏ thành nền tảng thử nghiệm cho firewall, routing, VPN, QoS, monitoring, IoT gateway hoặc các dịch vụ mạng riêng.

Về mặt Embedded Linux, OpenWrt là một ví dụ thực tế rất phù hợp vì nó thể hiện đầy đủ các thành phần của một hệ thống nhúng chạy Linux: bootloader, kernel, root filesystem, init system, cấu hình mạng, package manager và các daemon user space. Khi làm việc với OpenWrt, người học không chỉ chạy ứng dụng C mà còn phải quan tâm đến cách ứng dụng được đóng gói, cách service được khởi động, cách cấu hình được lưu trong hệ thống và cách thiết bị giao tiếp với mạng.

14.2 Kiến trúc phần mềm của OpenWrt

Một hệ thống OpenWrt thường có kernel Linux được cấu hình cho phần cứng mục tiêu, root filesystem tối giản và tập các dịch vụ mạng cần thiết. OpenWrt sử dụng BusyBox để cung cấp nhiều lệnh Unix cơ bản trong một binary nhỏ, phù hợp với thiết bị hạn chế bộ nhớ. Hệ thống cấu hình của OpenWrt thường xoay quanh UCI (Unified Configuration Interface), cho phép cấu hình network, wireless, firewall, DHCP hoặc các service khác theo một dạng thống nhất.

OpenWrt cũng có hệ thống quản lý gói thông qua `opkg`. Nhờ `opkg`, người dùng có thể cài thêm hoặc gỡ bỏ các package phù hợp với dung lượng flash và RAM của thiết bị. Đối với lập trình viên, điều này có nghĩa là ứng dụng tự viết có thể được đóng gói thành package, khai báo phụ thuộc thư viện và cài đặt lên thiết bị theo cách có kiểm soát thay vì copy thủ công từng file.

Ở tầng khởi động dịch vụ, OpenWrt thường dùng `procd` làm init và process supervisor. `procd` chịu trách nhiệm khởi động service, theo dõi tiến trình, xử lý hotplug và phối hợp với các script trong hệ thống. Đây là điểm khác biệt đáng chú ý so với nhiều bản phân phối Linux desktop/server dùng `systemd`. Vì vậy, khi chuyển kiến thức Linux Programming sang OpenWrt, người học cần hiểu rằng chương trình C không chỉ được biên dịch đúng mà còn cần được tích hợp đúng vào cơ chế service của hệ thống.

14.3 Phát triển ứng dụng C trên OpenWrt

Phát triển ứng dụng C cho OpenWrt thường liên quan đến cross-compilation. Thiết bị OpenWrt có thể dùng kiến trúc MIPS, ARM, x86 hoặc một kiến trúc khác, trong khi máy phát triển thường là PC x86_64. Vì vậy, ứng dụng cần được biên dịch bằng toolchain phù hợp với target. OpenWrt Build System cung cấp môi trường để cấu hình target, chọn package, biên dịch kernel, root filesystem và tạo firmware image.

Khi viết ứng dụng C cho OpenWrt, các kiến thức Basic C và Linux Programming được sử dụng trực tiếp. Quá trình preprocessing, compilation, assembly và linking quyết định binary cuối cùng có chạy được trên target hay không. Memory layout và virtual memory giúp phân tích lỗi runtime trên thiết bị có RAM nhỏ. System call giúp ứng dụng đọc ghi file cấu hình, thao tác socket, giao tiếp với device node hoặc ghi log. Process, signal và IPC giúp ứng dụng hoạt động như một daemon ổn định. Socket programming đặc biệt quan trọng vì OpenWrt là nền tảng mạng; nhiều ứng dụng trên OpenWrt cần mở TCP server, gửi UDP packet, giao tiếp HTTP, nhận dữ liệu từ LAN/WAN hoặc trao đổi thông tin với cloud server.

14.4 Liên hệ OpenWrt với boot sequence và hệ thống nhúng

OpenWrt cũng tuân theo logic chung của Embedded Linux boot sequence. Sau khi thiết bị được cấp nguồn, bootloader như U-Boot khởi tạo phần cứng ở mức cơ bản, nạp kernel Linux và truyền thông tin cần thiết cho kernel. Kernel khởi tạo driver, mount root filesystem và chuyển sang user space. Trong user space, init system của OpenWrt khởi động các service mạng, cấu hình interface, firewall, DHCP, DNS và các daemon khác.

Điểm đáng chú ý là OpenWrt thường chạy trên thiết bị có tài nguyên hạn chế, vì vậy mọi quyết định thiết kế đều phải cân nhắc dung lượng flash, RAM, thời gian boot và độ ổn định. Một ứng dụng C viết cho OpenWrt cần gọn, kiểm soát tài nguyên tốt, xử lý lỗi mạng cẩn thận và tích hợp đúng với cơ chế cấu hình/service của hệ thống. Do đó, OpenWrt là môi trường thực hành tốt để kết nối các kiến thức đã học: từ biên dịch C, linking, Linux system call, socket programming, process management cho đến boot sequence và triển khai phần mềm trên thiết bị nhúng.

15 Boot sequence, bootloader và kiến trúc ARM trong Embedded Linux

15.1 Vai trò của bootloader trong hệ thống nhúng

Trong một hệ thống Embedded Linux, bootloader là thành phần đứng giữa phần cứng vừa được cấp nguồn và Linux Kernel. Nhiệm vụ trực tiếp của bootloader là đưa hệ thống từ trạng thái rất tối thiểu sau reset đến trạng thái đủ điều kiện để kernel có thể chạy. Có thể xem bootloader là chương trình chịu trách nhiệm “đánh thức” phần cứng ở mức cơ bản, nạp kernel vào RAM và tạo môi trường thực thi ban đầu cho kernel.

Khi những dòng lệnh đầu tiên của bootloader được thực thi, hệ thống chưa ở trạng thái sẵn sàng như khi Linux đã chạy. DRAM thường chưa được cấu hình nên bộ nhớ chính chưa thể truy cập được. Các giao tiếp lưu trữ như NAND, SPI flash, eMMC, SD card hoặc USB cũng chưa chắc đã hoạt động. Thông thường chỉ có một lõi CPU chạy ở mức cấu hình an toàn, một lượng nhỏ SRAM bên trong SoC và boot ROM là có thể sử dụng. Vì vậy, bootloader không cần khởi tạo toàn bộ phần cứng như kernel, mà chỉ cần khởi tạo vừa đủ những thành phần cần thiết để nạp kernel, initramfs và Device Tree vào bộ nhớ.

Ngoài nhiệm vụ nạp kernel, bootloader còn cung cấp chế độ bảo trì hệ thống. Trên nhiều board ARM, U-Boot cho phép người phát triển dừng quá trình autoboot qua serial console, nhập lệnh để kiểm tra RAM, xem biến môi trường, nạp image qua TFTP, đọc image từ SD/eMMC/NAND, ghi image vào flash hoặc thay đổi cấu hình boot. Chế độ này đặc biệt quan trọng trong giai đoạn bring-up board, debug lỗi khởi động và cập nhật phần mềm nhúng.

15.2 Đặc điểm boot trên kiến trúc ARM

Trong thế giới PC, firmware như BIOS hoặc UEFI cung cấp một môi trường tương đối chuẩn để tìm và chạy bootloader. Ngược lại, trên ARM và các SoC nhúng, quá trình boot phụ thuộc rất mạnh vào từng họ chip và từng thiết kế board. Mỗi SoC thường có boot ROM riêng, thứ tự tìm nguồn boot riêng, yêu cầu định dạng image riêng và địa chỉ nạp riêng. Vì vậy, boot sequence trên ARM thường được tổ chức thành nhiều giai đoạn nhỏ thay vì chỉ có một bootloader duy nhất.

Trên ARM hiện đại, kernel không nên chứa cứng toàn bộ mô tả phần cứng của từng board. Thay vào đó, bootloader thường truyền cho kernel một Device Tree Blob. Device Tree mô tả các thành phần như CPU, RAM, bus, interrupt controller, UART, I2C, SPI, GPIO, Ethernet, flash và nhiều ngoại vi khác. Cách làm này giúp một kernel binary có thể hỗ trợ nhiều board khác nhau, miễn là kernel nhận được đúng Device Tree tương ứng. Đây là điểm rất quan trọng trong kiến trúc ARM, vì số lượng SoC và board ARM rất lớn, không đồng nhất như nền tảng PC.

Trước khi Device Tree được sử dụng rộng rãi, một số hệ thống ARM cũ truyền thông tin sang kernel bằng ATAGS hoặc machine number. Cách này có nhiều hạn chế vì lượng thông tin truyền được ít, nhiều chi tiết phần cứng phải hard-code trong kernel dưới dạng platform data. Từ các kernel ARM hiện đại, Device Tree trở thành phương pháp phổ biến hơn để tách mô tả phần cứng ra khỏi kernel, giúp việc port Linux sang board mới linh hoạt hơn.

15.3 Phase 1: ROM code trong SoC

Giai đoạn đầu tiên của boot sequence trên thiết bị nhúng là ROM code. Đây là đoạn mã nằm sẵn bên trong SoC, được nạp vào chip từ quá trình sản xuất và chạy ngay sau reset hoặc power-on. Vì lúc này chưa có bộ nhớ ngoài đáng tin cậy, mã đầu tiên bắt buộc phải nằm trong chip. ROM code thường là mã đóng của nhà sản xuất SoC và người phát triển không thể thay thế bằng mã nguồn mở.

ROM code thường không khởi tạo DRAM vì cấu hình DRAM phụ thuộc rất nhiều vào thiết kế board, loại RAM, tần số, timing và sơ đồ kết nối. Thay vào đó, ROM code sử dụng SRAM nội bộ của SoC. SRAM này có dung lượng nhỏ, có thể chỉ vài KB đến vài trăm KB, nhưng đủ để chứa một chương trình nạp thứ cấp nhỏ. ROM code sẽ tìm image ở các vị trí đã được SoC định nghĩa trước. Ví dụ, một số SoC có thể thử đọc từ các trang đầu của NAND flash, từ SPI flash, từ sector đầu của MMC/eMMC/SD card hoặc từ một file có tên cố định như `MLO` trên phân vùng boot.

Nếu các nguồn boot thông thường thất bại, một số SoC có thể chuyển sang cơ chế nhận dữ liệu qua USB, UART hoặc Ethernet. Cơ chế này rất hữu ích trong sản xuất, nạp firmware lần đầu hoặc phục hồi thiết bị bị lỗi flash. Tuy nhiên, trong quá trình vận hành bình thường, hệ thống thường boot từ flash hoặc thẻ nhớ đã được chuẩn bị sẵn. Khi ROM code nạp xong chương trình nhỏ vào SRAM, nó nhảy đến địa chỉ bắt đầu của chương trình đó. Chương trình này thường là SPL.

15.4 Phase 2: SPL và khởi tạo DRAM

SPL, viết tắt của Secondary Program Loader, là giai đoạn trung gian giữa ROM code và bootloader đầy đủ. Lý do cần SPL là SRAM nội bộ thường quá nhỏ để chứa toàn bộ U-Boot hoặc một bootloader có đầy đủ driver, filesystem và giao diện dòng lệnh. SPL vì vậy được thiết kế tối giản, chỉ chứa những chức năng cần thiết nhất để đưa hệ thống đến trạng thái có thể nạp chương trình lớn hơn vào DRAM.

Nhiệm vụ quan trọng nhất của SPL là khởi tạo bộ điều khiển DRAM. Việc này bao gồm cấu hình clock, PLL, nguồn, timing, bus width và các thanh ghi của memory controller. Nếu cấu hình DRAM sai, hệ thống có thể treo rất sớm, log không đầy đủ hoặc lỗi ngẫu nhiên khi nạp image lớn. Ngoài DRAM, SPL có thể khởi tạo thêm UART để in log, watchdog, MMC/eMMC, NAND, SPI flash hoặc driver filesystem đơn giản để đọc file như `u-boot.img`.

Thông thường SPL không cung cấp giao diện tương tác cho người dùng. Nó có thể in một số dòng log như phiên bản, nguồn boot đang thử hoặc lỗi nạp image, nhưng nhiệm vụ chính vẫn là nạp TPL hoặc U-Boot đầy đủ vào DRAM. Ở cuối giai đoạn SPL, DRAM đã hoạt động và bootloader đầy đủ đã được đặt vào RAM. SPL sau đó chuyển quyền thực thi sang địa chỉ của bootloader đầy đủ.

15.5 Phase 3: TPL hoặc U-Boot đầy đủ

Sau khi DRAM được khởi tạo, hệ thống có thể chạy một bootloader hoàn chỉnh hơn, thường là U-Boot. Giai đoạn này đôi khi được gọi là TPL, tức Tertiary Program Loader, nếu hệ thống phân chia rõ ROM code, SPL và bootloader đầy đủ. Khác với SPL, U-Boot đầy đủ có thể chứa nhiều driver hơn, hỗ trợ filesystem, network, command line, biến môi trường và nhiều định dạng image

khác nhau.

U-Boot có thể nạp kernel từ nhiều nguồn. Trong giai đoạn phát triển, kernel thường được nạp qua mạng bằng TFTP để giảm thời gian thử nghiệm. Trên thiết bị triển khai thực tế, kernel thường được đọc từ NAND flash, NOR flash, SPI flash, eMMC, SD card hoặc một phân vùng boot riêng. U-Boot cũng có thể nạp initramfs và Device Tree vào các địa chỉ RAM riêng. Sau khi các thành phần này đã nằm trong bộ nhớ, U-Boot gọi lệnh boot thích hợp để chuyển quyền điều khiển sang kernel.

Một điểm quan trọng của U-Boot là hệ thống biến môi trường. Các biến như địa chỉ nạp kernel, địa chỉ Device Tree, tên file image, địa chỉ IP của board, địa chỉ server TFTP, tham số kernel command line và script `bootcmd` quyết định hệ thống sẽ boot như thế nào. Khi bật nguồn, nếu người dùng không dùng autoboot trong thời gian `bootdelay`, U-Boot sẽ chạy nội dung `bootcmd`. Nhờ đó, quá trình boot có thể tự động nhưng vẫn cho phép can thiệp thủ công khi cần debug.

15.6 Nạp kernel, initramfs và Device Tree

Ở cuối giai đoạn bootloader, kernel image đã phải nằm trong RAM tại một địa chỉ phù hợp. Trên ARM, kernel có thể ở dạng `zImage`, `Image`, `uImage` hoặc FIT image tùy cấu hình. `zImage` là kernel đã nén và có đoạn tự giải nén. `uImage` là image có thêm header của U-Boot, chứa thông tin như kiến trúc, hệ điều hành, loại image, kiểu nén, địa chỉ load và entry point. FIT image là định dạng linh hoạt hơn, có thể đóng gói kernel, Device Tree, ramdisk và chữ ký xác thực trong một cấu trúc duy nhất.

Ngoài kernel, bootloader thường nạp Device Tree Blob, tức file `.dtb`, vào RAM. Device Tree được viết ở dạng source `.dts` hoặc `.dtsi` và được biên dịch bằng Device Tree Compiler thành `.dtb`. Kernel đọc `.dtb` để biết phần cứng nào tồn tại, địa chỉ thanh ghi của thiết bị, interrupt nào được dùng, clock nào liên quan và driver nào có thể bind với thiết bị thông qua thuộc tính `compatible`. Nếu Device Tree sai, kernel có thể vẫn boot nhưng thiếu thiết bị, không nhận UART, không mount được storage hoặc driver không probe được.

Initramfs là thành phần tùy chọn nhưng rất hữu ích. Nó là một root filesystem tạm thời nằm trong RAM, chứa các công cụ và module cần thiết cho giai đoạn đầu của kernel. Trong một số hệ thống, initramfs được dùng để nạp driver storage, giải mã root filesystem, kiểm tra phân vùng hoặc chọn rootfs thật. Với hệ thống nhúng nhỏ, có thể không cần initramfs nếu kernel có đủ driver built-in để mount root filesystem trực tiếp.

15.7 Handoff từ bootloader sang Linux Kernel

Khi bootloader chuyển quyền điều khiển cho Linux Kernel, nó không chỉ nhảy đến entry point của kernel. Nó còn phải truyền cho kernel các thông tin tối thiểu để kernel có thể tiếp tục quá trình khởi động. Những thông tin này gồm vị trí và kích thước RAM vật lý, kernel command line, địa chỉ Device Tree Blob, địa chỉ và kích thước initramfs nếu có, và trong các nền tảng cũ có thể gồm machine number hoặc ATAGS.

Kernel command line là một chuỗi ASCII điều khiển hành vi khởi động của Linux. Các tham số phổ biến gồm `console=` để chọn console log, `root=` để chỉ định root filesystem, `rootwait` để chờ

thiết bị lưu trữ xuất hiện, `init=` để chọn chương trình init, hoặc các tham số debug như `loglevel`. Trong Embedded Linux, một lỗi nhỏ ở command line cũng có thể làm kernel boot được nhưng không mount được root filesystem, dẫn đến kernel panic ở cuối quá trình khởi động.

Sau khi nhận quyền điều khiển, kernel giải nén nếu cần, thiết lập môi trường thực thi, khởi tạo MMU, scheduler, interrupt, memory management và các driver cơ bản. Kernel phân tích Device Tree để tạo danh sách thiết bị nền tảng, probe driver tương ứng, mount root filesystem và chạy chương trình init đầu tiên trong user space. Từ thời điểm init chạy, bootloader không còn vai trò trong hệ thống và vùng nhớ mà bootloader từng sử dụng có thể được kernel tái sử dụng.

15.8 Falcon mode và tối ưu thời gian boot

Trong một số hệ thống nhúng, thời gian khởi động là yêu cầu quan trọng. Quy trình thông thường ROM code nạp SPL, SPL nạp U-Boot đầy đủ, U-Boot chạy script rồi nạp kernel có thể linh hoạt nhưng mất thêm thời gian. U-Boot hỗ trợ một hướng tối ưu gọi là Falcon mode. Ý tưởng của Falcon mode là để SPL nạp trực tiếp Linux Kernel từ một vị trí đã biết, truyền sẵn tham số cần thiết và bỏ qua giai đoạn U-Boot đầy đủ.

Falcon mode giúp giảm số bước boot, giảm thời gian chờ autoboot và loại bỏ tương tác dòng lệnh trong quá trình khởi động. Đổi lại, hệ thống mất đi sự linh hoạt của U-Boot đầy đủ trong mỗi lần boot. Vì vậy, Falcon mode phù hợp với sản phẩm ổn định, có quy trình update rõ ràng và không cần can thiệp debug thường xuyên. Trong giai đoạn phát triển, người ta vẫn thường giữ U-Boot đầy đủ để dễ nạp image, thay đổi command line và kiểm tra thiết bị.

15.9 Tổng kết luồng boot trên ARM Embedded Linux

Tổng thể, boot sequence trên một board ARM chạy Embedded Linux có thể được hiểu theo chuỗi sau. Sau reset, CPU bắt đầu từ boot ROM trong SoC. ROM code chọn nguồn boot và nạp SPL vào SRAM. SPL khởi tạo DRAM và các giao tiếp tối thiểu rồi nạp U-Boot đầy đủ vào DRAM. U-Boot cung cấp môi trường dòng lệnh, đọc biến môi trường, nạp kernel, Device Tree và initramfs từ flash, thẻ nhớ hoặc mạng. Sau đó U-Boot truyền command line, Device Tree và các thông tin cần thiết cho kernel. Kernel tiếp quản hệ thống, khởi tạo driver, mount root filesystem và chạy init trong user space.

Cách nhìn này cho thấy bootloader không chỉ là một chương trình nạp kernel đơn giản. Nó là thành phần quyết định khả năng bring-up board, cập nhật firmware, debug lỗi boot, lựa chọn image, truyền mô tả phần cứng và đảm bảo kernel bắt đầu chạy trong trạng thái hợp lệ. Đối với người học Embedded Linux, hiểu boot sequence và bootloader trên ARM là điều kiện quan trọng để tiếp tục học U-Boot, Device Tree, kernel configuration, root filesystem và cơ chế cập nhật phần mềm trên thiết bị thật.

16 Ứng dụng kiến thức vào MediaTek AP 6

16.1 Bối cảnh áp dụng trên thiết bị Access Point

MediaTek AP 6 có thể được xem như một ví dụ thực tế để liên hệ các nội dung đã học về Embedded Linux, OpenWrt, socket programming và bootloader. Khác với một chương trình C chạy độc lập trên PC, phần mềm trên một thiết bị Access Point phải hoạt động trong một hệ thống nhúng hoàn chỉnh. Hệ thống đó có bootloader để khởi động kernel, root filesystem chứa các dịch vụ user space, driver Wi-Fi giao tiếp với phần cứng radio, các daemon cấu hình mạng và giao diện quản lý để người dùng thay đổi tham số vận hành.

Trong môi trường AP, yêu cầu quan trọng không chỉ là thiết bị boot được vào Linux mà còn phải tạo được mạng Wi-Fi ổn định, phát SSID đúng cấu hình, chọn channel phù hợp, áp dụng bảo mật, quản lý client và duy trì kết nối mạng trong thời gian dài. Vì vậy, các kiến thức về process, thread, IPC, signal, socket và OpenWrt đều có thể được áp dụng trực tiếp. Một daemon cấu hình Wi-Fi có thể nhận lệnh từ giao diện web hoặc từ một service quản lý, chuyển đổi cấu hình đó thành profile của driver, gọi lệnh hệ thống hoặc ioctl để cập nhật tham số, sau đó kiểm tra trạng thái hoạt động của interface.

16.2 Cấu hình Wi-Fi SoftAP bằng profile và iwpriv

Theo guide MediaTek Wi-Fi SoftAP, driver hỗ trợ hai hướng cấu hình chính: cấu hình bằng profile và cấu hình động bằng lệnh `iwpriv`. Cấu hình bằng profile phù hợp với các tham số cần có ngay khi driver hoặc interface được khởi tạo. Profile có thể chứa các trường như country region, country code, số lượng BSSID, SSID, wireless mode, channel, beacon period, DTIM period, transmit power, WMM, ẩn SSID, cơ chế chống flooding, forwarding giữa các BSSID và nhiều tham số khác. Đây là dạng cấu hình gần với cách một firmware AP lưu thông tin mặc định trước khi đưa Wi-Fi lên hoạt động.

Ngược lại, `iwpriv` phù hợp với việc thay đổi hoặc kiểm tra tham số khi hệ thống đang chạy. Ví dụ, người phát triển có thể dùng `iwpriv` để đổi SSID, chỉnh channel, kiểm tra thông tin driver, bật tắt một số tính năng hoặc debug hiệu năng TX/RX. Về mặt Linux Programming, các thao tác này có thể được hiểu là cách user space gửi yêu cầu xuống driver thông qua wireless extension hoặc ioctl. Điều này liên hệ trực tiếp với phần system call đã học: ứng dụng user space không điều khiển phần cứng Wi-Fi trực tiếp mà phải đi qua kernel và driver.

Trên một nền tảng giống OpenWrt, các tham số từ UCI hoặc file cấu hình hệ thống có thể được chuyển đổi thành profile MediaTek hoặc thành các lệnh runtime tương ứng. Khi người dùng thay đổi SSID, mật khẩu hoặc channel từ giao diện quản lý, backend có thể cập nhật file cấu hình, restart service Wi-Fi hoặc gọi công cụ điều khiển driver. Như vậy, kiến thức về file I/O, process, signal và service management đều tham gia vào quá trình cấu hình SoftAP.

16.3 Các nhóm tham số quan trọng khi triển khai AP 6

Khi triển khai trên MediaTek AP 6, nhóm tham số đầu tiên cần chú ý là tham số vùng quốc gia và kênh tần số. Các trường như `CountryRegion`, `CountryRegionABand`, `CountryCode`, `ChannelGeography`, `Channel` và `AutoChannelSelect` ảnh hưởng trực tiếp đến danh sách channel hợp lệ và hành vi chọn kênh. Đây là nhóm cấu hình quan trọng vì thiết bị Wi-Fi phải tuân thủ quy định tần số của từng khu vực. Nếu cấu hình sai country code hoặc channel, thiết bị có thể không phát đúng băng tần mong muốn, không lên được interface hoặc vi phạm quy định sử dụng phổ tần.

Nhóm tham số thứ hai liên quan đến nhận dạng mạng và chế độ Wi-Fi. `SSID` quyết định tên mạng hiển thị cho client, `BssidNum` liên quan đến số lượng SSID/BSSID, còn `WirelessMode` quyết định chuẩn hoạt động như 802.11b/g/n/ac tùy chip và driver hỗ trợ. Với một AP nhiều SSID, các cơ chế như MBSSID và forwarding giữa các BSSID trở nên quan trọng vì chúng quyết định client ở các SSID khác nhau có được giao tiếp với nhau hay không. Các tham số như `NoForwarding`, `NoForwardingBTNBSSID` và `NoForwardingMBCast` có thể được dùng để cô lập client hoặc cô lập lưu lượng giữa các mạng Wi-Fi khác nhau.

Nhóm tham số thứ ba liên quan đến hiệu năng và chất lượng dịch vụ. Các trường như `BeaconPeriod`, `DtimPeriod`, `TxPower`, `BasicRate`, `SupportRate`, `SupportHTRate`, `WmmCapable` và `TxBurst` ảnh hưởng đến vùng phủ sóng, độ tương thích client, độ trễ và cân bằng giữa TX/RX. Trong quá trình test AP, nếu thấy throughput không cân bằng hoặc client kết nối không ổn định, người phát triển cần kiểm tra các tham số này cùng log driver. Đây là nơi kiến thức về debug Linux, đọc log, dùng command line và phân tích trạng thái runtime trở nên cần thiết.

Nhóm tham số thứ tư liên quan đến bảo mật và độ ổn định. Một AP thực tế cần cấu hình authentication, encryption, WPS, PMF, ACL và các ngưỡng chống flooding như `AuthFloodThreshold`, `AssocReqFloodThreshold`, `ReassocReqFloodThreshold` hoặc `DeauthFloodThreshold`. Những tham số này giúp AP kiểm soát hành vi client và giảm rủi ro bị tấn công ở tầng Wi-Fi management frame. Đối với sản phẩm thương mại, bảo mật Wi-Fi không thể chỉ dừng ở việc đặt mật khẩu mà còn phải tính đến khả năng chống lạm dụng và khả năng vận hành ổn định trong môi trường nhiều client.

16.4 Liên hệ với socket programming và daemon quản lý

Trong một thiết bị AP 6, socket programming thường được dùng ở tầng quản lý và điều khiển. Một daemon user space có thể mở TCP socket hoặc Unix domain socket để nhận lệnh cấu hình từ web UI, CLI, mobile app hoặc cloud service. Sau khi nhận yêu cầu, daemon kiểm tra dữ liệu, cập nhật cấu hình, gọi script hệ thống hoặc dùng `ioctl` để thay đổi tham số driver. Với các tác vụ giám sát, daemon có thể gửi log, trạng thái client, số lượng station, channel hiện tại hoặc thông tin throughput về server quản lý thông qua TCP/UDP socket.

Cách tổ chức này cho thấy mối liên hệ giữa socket programming và các kiến thức khác trong báo cáo. Process và thread giúp daemon xử lý nhiều kết nối hoặc nhiều tác vụ đồng thời. Signal giúp service thoát an toàn khi hệ thống restart Wi-Fi hoặc shutdown. IPC giúp các tiến trình như web backend, network manager và Wi-Fi manager trao đổi trạng thái. File I/O giúp đọc ghi profile hoặc UCI config. System call và `ioctl` là cầu nối để user space tác động đến driver. Nếu daemon không kiểm soát tốt tài nguyên, thiết bị có thể bị leak file descriptor, treo service hoặc mất khả năng cấu

hình Wi-Fi sau thời gian chạy dài.

16.5 Quy trình áp dụng vào bring-up và kiểm thử

Khi bring-up MediaTek AP 6, quá trình thường bắt đầu từ bootloader và kernel. Thiết bị phải boot được vào Linux, nhận đúng flash, Ethernet, UART và Wi-Fi chipset. Sau đó, driver Wi-Fi cần được nạp thành công, interface xuất hiện đúng tên, profile SoftAP được đọc và SSID được phát ra. Nếu AP không phát Wi-Fi, người phát triển cần kiểm tra theo từng lớp: boot log, kernel log, module driver, firmware Wi-Fi, file profile, quyền truy cập file, script khởi động service và lệnh cấu hình runtime.

Sau khi interface hoạt động, bước tiếp theo là kiểm thử cấu hình cơ bản như SSID, channel, country code, security mode và DHCP/network bridge. Tiếp đó là kiểm thử nhiều client, roaming, throughput, độ ổn định khi restart Wi-Fi, thay đổi cấu hình runtime và phục hồi sau lỗi. Với các tính năng nâng cao như ACS, MBSSID, WPS, PMF, ACL hoặc IGMP snooping, cần có test case riêng để xác nhận tham số trong profile thực sự được driver áp dụng. Các công cụ như log hệ thống, lệnh `iwpriv`, script kiểm tra trạng thái, packet capture và chương trình socket test đều có thể hỗ trợ quá trình này.

Nhìn chung, MediaTek AP 6 là một môi trường ứng dụng rất phù hợp cho các kiến thức đã học trong kỳ thực tập. Basic C giúp hiểu cách viết và build công cụ quản lý. Linux Programming giúp hiểu cách daemon tương tác với kernel và driver. OpenWrt giúp tổ chức package, service và cấu hình hệ thống. Socket programming giúp xây dựng kênh điều khiển và giám sát. Boot sequence giúp phân tích lỗi từ lúc cấp nguồn đến khi Wi-Fi service chạy. Nhờ đó, người học có thể nhìn thiết bị AP không chỉ như một router phát Wi-Fi, mà như một hệ thống Embedded Linux hoàn chỉnh có nhiều tầng phần mềm phối hợp với nhau.

17 Văn hóa công ty trong quá trình thực tập

17.1 Môi trường làm việc và tinh thần học hỏi

Trong thời gian thực tập tại Viettel, sinh viên được tiếp cận một môi trường làm việc có tính kỷ luật cao, coi trọng hiệu quả công việc và tinh thần chủ động học hỏi. Đặc điểm nổi bật của môi trường này là yêu cầu mỗi cá nhân không chỉ hoàn thành phần việc được giao, mà còn phải hiểu bản chất vấn đề, biết đặt câu hỏi và có khả năng tự tìm hướng giải quyết trước khi trao đổi với người hướng dẫn. Điều này phù hợp với đặc thù của lĩnh vực Embedded Linux, nơi các lỗi kỹ thuật thường không thể xử lý bằng cách ghi nhớ máy móc mà cần phân tích từ nhiều lớp, từ mã nguồn C, thư viện, system call, kernel, bootloader cho đến phần cứng.

Văn hóa làm việc cũng nhấn mạnh tinh thần trách nhiệm với sản phẩm. Trong các bài học và trao đổi hằng ngày, sinh viên nhận thấy rằng một đoạn mã chạy được chưa đủ để được xem là hoàn thiện. Chương trình cần có khả năng xử lý lỗi, sử dụng tài nguyên hợp lý, dễ bảo trì và phù hợp với môi trường triển khai thực tế. Tư duy này giúp sinh viên thay đổi cách nhìn từ việc chỉ viết code để giải bài tập sang viết phần mềm có thể vận hành trong một hệ thống thật.

17.2 Văn hóa seminar và chia sẻ kiến thức cho intern

Một điểm đáng chú ý trong quá trình thực tập là văn hóa seminar dành cho intern. Các buổi seminar tạo ra không gian để sinh viên trình bày lại nội dung đã học, trao đổi khó khăn gặp phải và nhận góp ý từ người hướng dẫn hoặc các thành viên khác trong nhóm. Thay vì học một chiều, seminar buộc người học phải hệ thống hóa kiến thức, diễn đạt lại bằng ngôn ngữ của mình và trả lời các câu hỏi phản biện. Đây là cách học rất hiệu quả đối với các chủ đề nền tảng như Basic C, Linux Programming, socket programming, OpenWrt và boot sequence.

Thông qua seminar, sinh viên không chỉ củng cố kiến thức kỹ thuật mà còn rèn luyện kỹ năng trình bày. Khi giải thích một nội dung như virtual memory, process, thread hay IPC cho người khác, sinh viên phải hiểu rõ khái niệm, phân biệt được các trường hợp sử dụng và biết liên hệ với ví dụ thực tế. Những câu hỏi trong seminar cũng giúp phát hiện các điểm còn hiểu chưa sâu, từ đó định hướng lại việc tự học. Đối với một intern, đây là cơ hội tốt để chuyển từ trạng thái tiếp nhận kiến thức sang trạng thái chủ động phân tích và chia sẻ kiến thức.

Bên cạnh seminar kỹ thuật, văn hóa trao đổi trong nhóm cũng tạo điều kiện để intern làm quen với quy trình làm việc chuyên nghiệp. Các nội dung như sử dụng Git, viết báo cáo, đọc tài liệu, ghi chú kết quả thực nghiệm và trình bày tiến độ đều được xem là một phần của quá trình học. Điều này giúp sinh viên hiểu rằng năng lực kỹ thuật cần đi kèm với khả năng giao tiếp, phối hợp và báo cáo rõ ràng.

17.3 Tác phong và bài học cá nhân

Quá trình thực tập giúp sinh viên nhận ra tầm quan trọng của tác phong làm việc nghiêm túc. Trong môi trường doanh nghiệp công nghệ, thời gian, chất lượng và tính chính xác đều có ý nghĩa quan trọng. Khi tìm hiểu một vấn đề, sinh viên cần ghi lại nguồn tài liệu, các bước đã thử, lỗi gặp phải và kết quả thu được. Cách làm này giúp quá trình trao đổi với người hướng dẫn hiệu quả hơn, đồng thời tạo thói quen làm việc có bằng chứng thay vì chỉ dựa trên cảm tính.

Một bài học khác là tinh thần tự học. Các chủ đề như Embedded Linux, OpenWrt hay socket programming có phạm vi rộng, không thể nắm vững chỉ qua một buổi hướng dẫn. Người học cần chủ động đọc tài liệu, chạy thử ví dụ, quan sát log, phân tích lỗi và tự tổng hợp lại kiến thức. Văn hóa công ty khuyến khích intern đặt câu hỏi, nhưng cũng yêu cầu câu hỏi phải đi kèm với quá trình tự tìm hiểu trước đó. Nhờ vậy, sinh viên dần hình thành tư duy độc lập hơn khi tiếp cận vấn đề kỹ thuật.

Nhìn chung, văn hóa thực tập tại Viettel không chỉ giúp sinh viên bổ sung kiến thức chuyên môn mà còn giúp rèn luyện phong cách làm việc. Sự kết hợp giữa seminar, hướng dẫn kỹ thuật, tự nghiên cứu và báo cáo tiến độ tạo ra môi trường phù hợp để sinh viên chuẩn bị cho công việc phát triển phần mềm nhúng trong thực tế.

18 Nhật ký công việc

Phần này được để trống để sinh viên bổ sung nhật ký công việc sau quá trình thực tập. Nội dung có thể ghi theo từng ngày hoặc từng tuần, bao gồm công việc đã thực hiện, kiến thức đã học, lỗi

gặp phải, cách khắc phục và kết quả đạt được.

Tuần 1:

Tuần 2:

Tuần 3:

Tuần 4:

19 Kết quả đạt được

Sau quá trình thực tập, sinh viên đã hệ thống hóa được các kiến thức nền tảng cần thiết cho Embedded Linux. Trước hết, sinh viên hiểu được một chương trình C được tạo ra như thế nào, từ mã nguồn ban đầu đến executable hoặc ELF cuối cùng. Điều này giúp việc đọc log build, phân tích lỗi compiler, linker và lỗi thiếu thư viện trở nên rõ ràng hơn. Tiếp theo, sinh viên nắm được tổ chức bộ nhớ của process, vai trò của heap, stack, text, data, BSS và vùng mmap, từ đó có cơ sở phân tích các lỗi bộ nhớ thường gặp trong C.

Về Linux Programming, sinh viên hiểu được ranh giới giữa user space và kernel space, vai trò của system call, cơ chế virtual memory, cách Linux quản lý process và thread, cũng như các phương thức IPC và signal. Việc tìm hiểu thêm kiến trúc UNIX giúp sinh viên liên kết file descriptor, VFS, inode, process context, cache và interrupt thành một mô hình thống nhất. Phần device driver giúp hình dung đường đi từ lời gọi user space qua VFS hoặc network stack xuống callback driver, thành ghi I/O, DMA và phần cứng. Đây là các kiến thức trực tiếp phục vụ việc viết và debug phần mềm nhúng chạy lâu dài. Ngoài ra, sinh viên cũng nắm được vai trò của Git/GitLab workflow trong quản lý mã nguồn và phối hợp phát triển phần mềm.

Một kết quả quan trọng khác là sinh viên đã liên hệ được các nội dung trên với boot sequence của Linux, đồng thời bước đầu làm quen với văn hóa seminar, cách báo cáo tiến độ và tác phong làm việc trong môi trường doanh nghiệp. Việc hiểu ROM code, SPL, U-Boot, kernel, Device Tree, root filesystem và systemd giúp hình thành cái nhìn tổng thể về vòng đời của một hệ thống Embedded Linux, từ lúc cấp nguồn đến khi ứng dụng user space hoạt động.

20 Khó khăn và hướng khắc phục

Khó khăn lớn nhất trong quá trình học là các khái niệm không tồn tại độc lập. Virtual memory liên quan đến cả phần cứng MMU, page table và chính sách của kernel. Process và thread liên quan đến scheduler, tài nguyên hệ thống và đồng bộ dữ liệu. IPC liên quan đến cả hiệu năng lẫn an toàn truy cập. Boot sequence lại yêu cầu hiểu thêm đặc điểm phần cứng của SoC, SRAM, DRAM và thiết bị lưu trữ. Vì vậy, nếu chỉ đọc lý thuyết, người học dễ nhớ khái niệm nhưng khó hình dung cách chúng hoạt động trong hệ thống thật.

Hướng khắc phục phù hợp là kết hợp đọc tài liệu với thực hành nhỏ. Người học có thể tự biên dịch chương trình C qua từng giai đoạn bằng `gcc -E`, `gcc -S`, `gcc -c` và linking. Có thể dùng `readelf`, `objdump` và `ldd` để quan sát executable và shared library. Các ví dụ nhỏ về `fork`, `pthread`, `pipe`, `mmap` và `sigaction` giúp biến khái niệm thành kinh nghiệm thực tế. Khi làm việc với board nhúng, việc đọc log boot từ U-Boot đến kernel và systemd cũng giúp liên hệ rõ ràng giữa lý thuyết và hệ thống thật.

21 Kết luận

Basic C và Linux Programming là nền tảng quan trọng để học Embedded Linux. Từ quá trình biên dịch C, người học hiểu cách source code trở thành chương trình chạy trên CPU. Từ memory layout và linking, người học hiểu cách chương trình được nạp, chia vùng bộ nhớ và phụ thuộc vào thư viện. Từ kiến trúc UNIX/Linux kernel, system call, VFS, inode và virtual memory, người học hiểu cách kernel tổ chức tài nguyên và tạo ranh giới giữa user space với kernel space. Từ process, thread, IPC và signal, người học có khả năng xây dựng ứng dụng Linux nhiều tác vụ, có đồng bộ và có giao tiếp dữ liệu. Từ device driver, người học hiểu thêm cách kernel quản lý thiết bị, interrupt, I/O memory, DMA và concurrency. Từ socket programming, OpenWrt và ứng dụng trên MediaTek AP 6, người học có thêm góc nhìn thực tế về phát triển ứng dụng mạng trên thiết bị nhúng. Từ Git workflow, người học có kỹ năng quản lý mã nguồn trong môi trường phát triển chuyên nghiệp.

Khi đặt trong bối cảnh Embedded Linux, các kiến thức này giúp giải thích cách ứng dụng user space giao tiếp với driver, cách daemon được systemd khởi động, cách chương trình được cross-compile, cách root filesystem chứa thư viện cần thiết và cách hệ thống đi từ bootloader đến kernel rồi đến user space. Đây là nền tảng cần thiết trước khi tiếp tục học các chủ đề nâng cao như U-Boot, Device Tree, Linux kernel driver, Buildroot, Yocto và tối ưu hệ thống nhúng.